

MLPy Workshop 7

March 10, 2026

1 Week 7 - Support Vector Machines

Student Name 1, Student Name 2

1.0.1 Aims

The main concepts covered in this notebook are:

- understanding separable vs non-separable data
- implementing SVMs
- use of different kernels and parameter tuning in SVMs

1. Setup
2. SVC
3. Model assessment
4. Spam Data

This week, we will be exploring the basics of support vector machine models. We will be focusing on support vector machines for classification, which is provided by sklearn in the **SVC** model. For more details, please see <https://scikit-learn.org/stable/modules/svm.html>

The main class that we are using is [sklearn.svm.SVC](#)

As usual, during workshops, you will complete the worksheets together in teams of 2-3, using **pair programming**. When completing worksheets:

- You will have tasks tagged by (CORE) and (EXTRA).
- Your primary aim is to complete the (CORE) components during the WS session, afterwards you can try to complete the (EXTRA) tasks for your self-learning process.

Instructions for submitting your workshops can be found at the end of worksheet. As a reminder, you must submit a pdf of your notebook on Learn by 16:00 PM on the Friday of the week the workshop was given.

2 Setup

2.1 Packages

Let's load the some of the packages needed for this workshop.

```
[1]: # Data libraries
import pandas as pd
import numpy as np

# Plotting libraries
import matplotlib.pyplot as plt
import seaborn as sns

# sklearn modules
import sklearn
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC # SVM
from sklearn.preprocessing import StandardScaler # scaling features
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import GridSearchCV, KFold, StratifiedKFold
```

2.2 Helper Functions

This helper function plots the data and visualized the decision boundary and margin.

```
[2]: from sklearn.inspection import DecisionBoundaryDisplay
import sklearn.preprocessing

# Visualize the decision boundary and margin
# For D=2 inputs and binary classification
def plot_margin(model, X, y, figsize=(8,7)):

    fig, ax = plt.subplots(1,1,figsize=figsize)

    # Scatter plot of the inputs colored by class
    ax.scatter(X[:,0], X[:,1], c=y, s=30)

    # Show decision boundary
    DecisionBoundaryDisplay.from_estimator(
        model,
        X,
        plot_method="contour",
        colors="k",
        levels=[-1, 0, 1],
        linestyles=["--", "-", "--"],
        ax=ax,
    )

    # Highlight support vectors
    # If pipeline with StandardScaler, inverse transform the support vectors
    if (isinstance(model, sklearn.pipeline.Pipeline)):
```

```

        if (isinstance(model[0], sklearn.preprocessing.StandardScaler)):
            support_vectors = model[0].inverse_transform(model[-1].
↪support_vectors_)
        else:
            support_vectors = model[-1].support_vectors_
    else:
        support_vectors = model.support_vectors_
    ax.scatter(
        support_vectors[:, 0],
        support_vectors[:, 1],
        s=100,
        linewidth=1,
        facecolors="none",
        edgecolors="k",
    )
    plt.show()

```

3 Support Vector Classifier

The class **SVC** implements support vector classifiers and support vector machines. When creating an SVC object, various options are available, including:

- **C**: the inverse regularization parameter. NOTE: this defaults to **C=1** but should always be tuned.
- **kernel**: options include the **linear** kernel (**linear**), **polynomial** kernel (**poly**), **radial basis function** kernel (**rbf**), **sigmoid** kernel (**sigmoid**), or a user-defined kernel.
- **degree**: degree if using the **polynomial** kernel
- **gamma**: kernel coefficient for **rbf**, **polynomial**, or **sigmoid** kernels.
- **coef0**: additional coefficient term for the **polynomial** or **sigmoid** kernels.

After calling `.fit()`, the SVC object will have a number of attributes including:

- **support_vectors_**: containing the support vectors.

To predict the class labels from the fitted SVC model, we can call `.predict()`. And although SVMs only provide class labels and not the corresponding class probabilities, SVC provides a method to estimate the class probabilities by calling `.predict_proba()` (or `.predict_log_proba()` on the log scale) using Platt scaling. This uses cross-validation and makes the implementation slower, thus to turn on this option of estimating the class probabilities, you must first set **probability=True** when creating the SVC object. Also note that these probability estimates are unreliable on small datasets.

More details on **kernels** are available here: <https://scikit-learn.org/stable/modules/svm.html#svm-kernels>

3.0.1 Difference between SVC and LinearSVC

Note the **sklearn** implements two linear support vector classification models **LinearSVC()** and **SVC(kernel='linear')**, which yield slightly different decision boundaries, due to the following differences:

- **LinearSVC** (based on LIBLINEAR) is **faster** than **SVC** (based on LIBSVM)
- **LinearSVC** minimizes the squared hinge loss while **SVC** minimizes the regular hinge loss.
- **LinearSVC** uses the One-vs-Rest scheme for multiclass classification while **SVC** uses the One-vs-One scheme for multiclass classification.
- **LinearSVC** does not provide some of the attributes of **SVC**, such as the support vectors.

For further details, see the documentations of the two classes

- **SVC** with `kernel=linear`: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>
- **LinearSVC**: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>

In the following, we will focus on the **SVC** class.

3.1 Linearly Separable Data

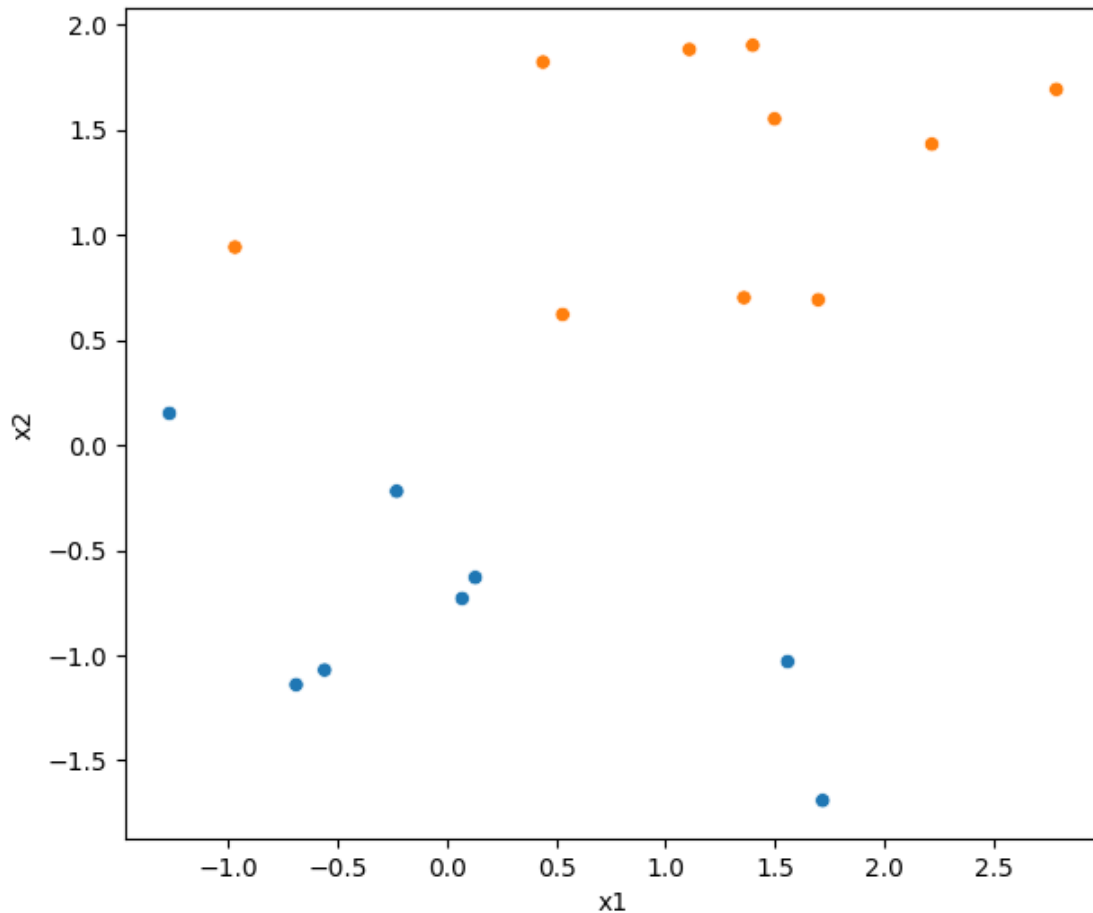
We will begin by examining various toy data sets to explore the basics of these models. For the first example, we will read in data from `ex1.csv`.

```
[3]: ex1 = pd.read_csv("ex1.csv")
      ex1.head()
```

```
[3]:      x1      x2  y
0 -0.56 -1.07  A
1 -0.23 -0.22  A
2  1.56 -1.03  A
3  0.07 -0.73  A
4  0.13 -0.63  A
```

Plotting the data below, we can see the that data is composed of two classes in two dimensions, and it is clear that these two classes are linearly separable.

```
[4]: plt.figure(figsize=(7,6))
      sns.scatterplot(x='x1', y='x2', hue='y', data=ex1, legend=False)
      plt.show()
```



Now, let separate the features and outcome and encode the outcome to a binary vector.

```
[5]: # Extracting the features and output and encoding y
X_ex1 = np.array(ex1.drop('y', axis=1))
y_ex1 = LabelEncoder().fit_transform(ex1.y)

print(X_ex1.shape)
print(y_ex1.shape)
```

```
(18, 2)
```

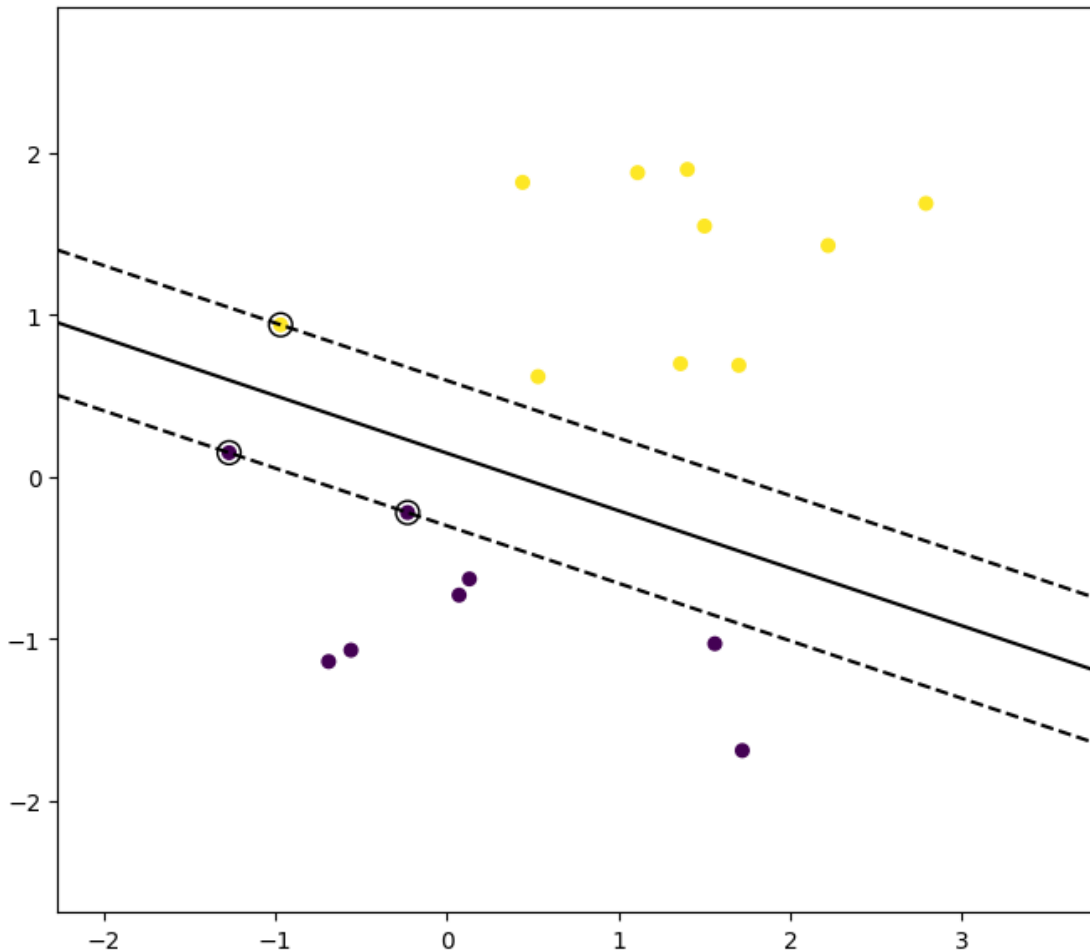
```
(18,)
```

3.1.1 Exercise 1 (CORE)

- Create and fit an SVC model using a **linear kernel** and **C=100**.
- Visualize the decision boundary, margin, and support vectors using the function `plot_margin` defined above. How many support vectors are there for each class? Are they on right side of the margin? hyperplane?

```
[6]: # Fit the SVC
svm = SVC(kernel='linear', C=100).fit(X_ex1, y_ex1)

# Visualize the decision boundary, margin and support vectors
plot_margin(svm, X_ex1, y_ex1)
```



In total there are three support vectors: - 1 support vector in the yellow class on the decision boundary, and - 2 support vectors in the purple class both on the decision boundary.

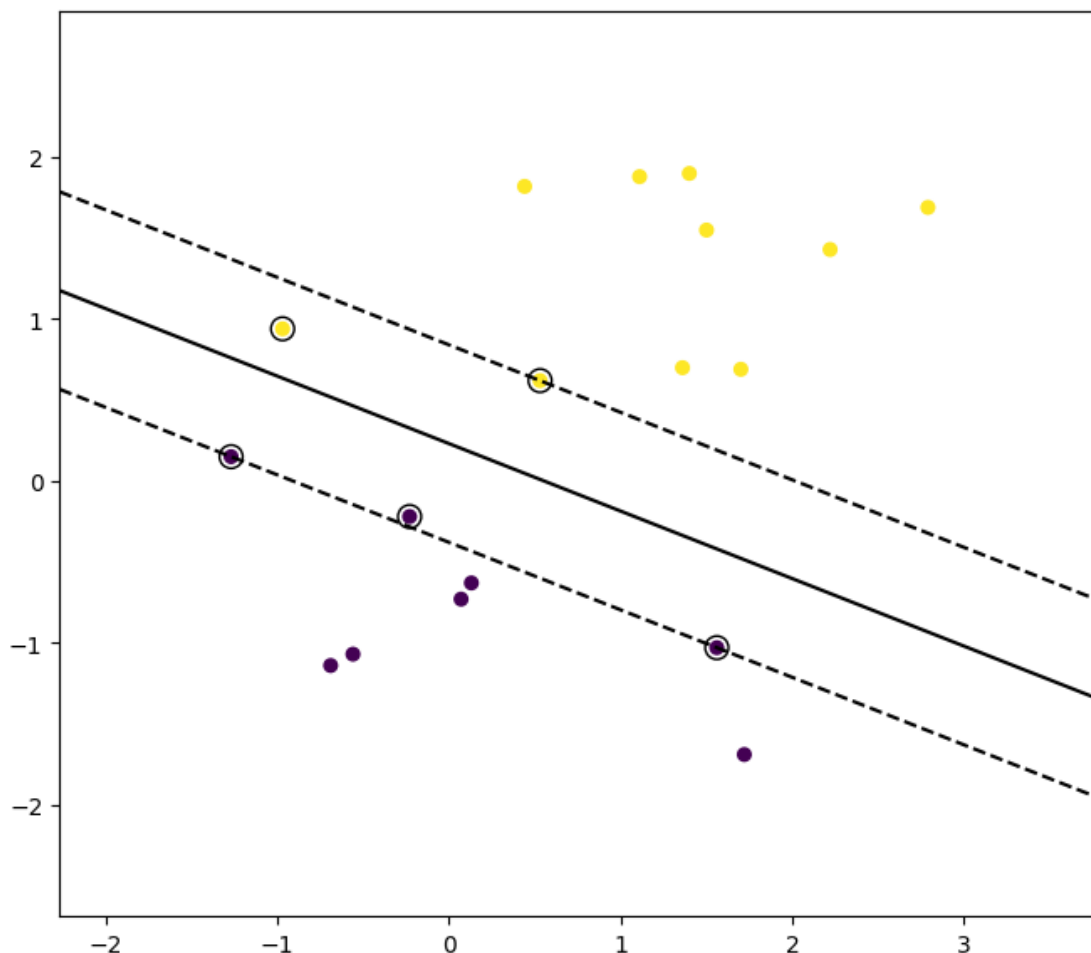
3.1.2 Exercise 2 (CORE)

Now, let's see how the results change with a small value of C .

- Create and fit an SVC model using a **linear kernel** and $C=1$.
- Visualize the decision boundary, margin, and support vectors. Now, how many support vectors are there for each class? Are they on right side of the margin? hyperplane? How has the margin changed?

```
[7]: # Fit the SVC with a smaller value of C
svm2 = SVC(kernel='linear', C=1).fit(X_ex1, y_ex1)

# Visualize the decision boundary, margin and support vectors
plot_margin(svm2, X_ex1, y_ex1)
```



Now, the margin is larger. A smaller value of C places more emphasis on minimizing the ℓ_2 norm of the weights at the cost of larger slack variables. This results in an increase in the number of support vectors (from 3 to 5 in total), and for each class: - there are two support vectors for the yellow class, one on the decision boundary and one on the wrong side of the margin but correct side of the hyperplane. - there are three support vectors for the purple class, two on the decision boundary, and one on the wrong side of the margin but correct side of the hyperplane.

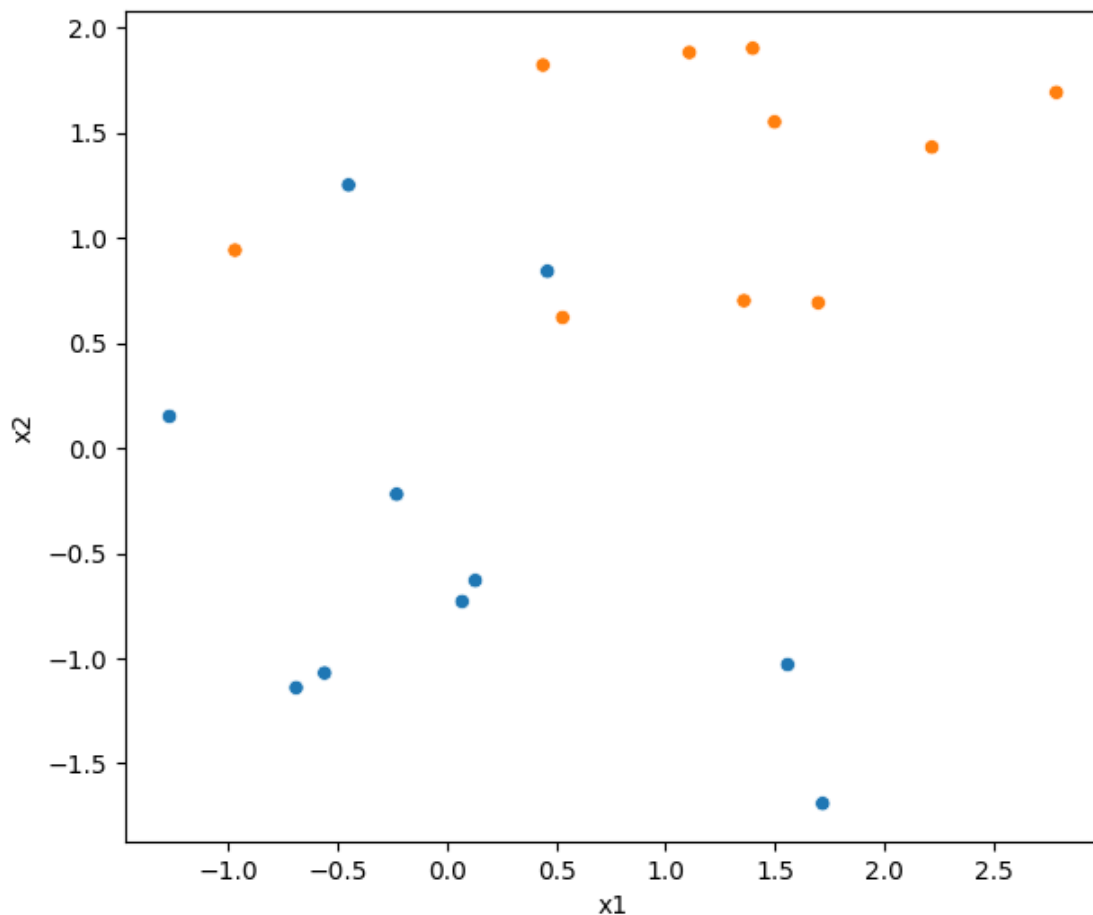
3.2 Non-Separable Data

Next, we complicate our previous example by adding two additional points from class A to our data, which result in data that are no longer linearly separable.

```
[8]: # Read in the data
ex2 = pd.read_csv("ex2.csv")

# Visualize the data
plt.figure(figsize=(7,6))
sns.scatterplot(x='x1', y='x2', hue='y', data=ex2, legend=False)
plt.show()

# Extracting the features and output and encoding y
X_ex2 = np.array(ex2.drop('y', axis=1))
y_ex2 = LabelEncoder().fit_transform(ex2.y)
```

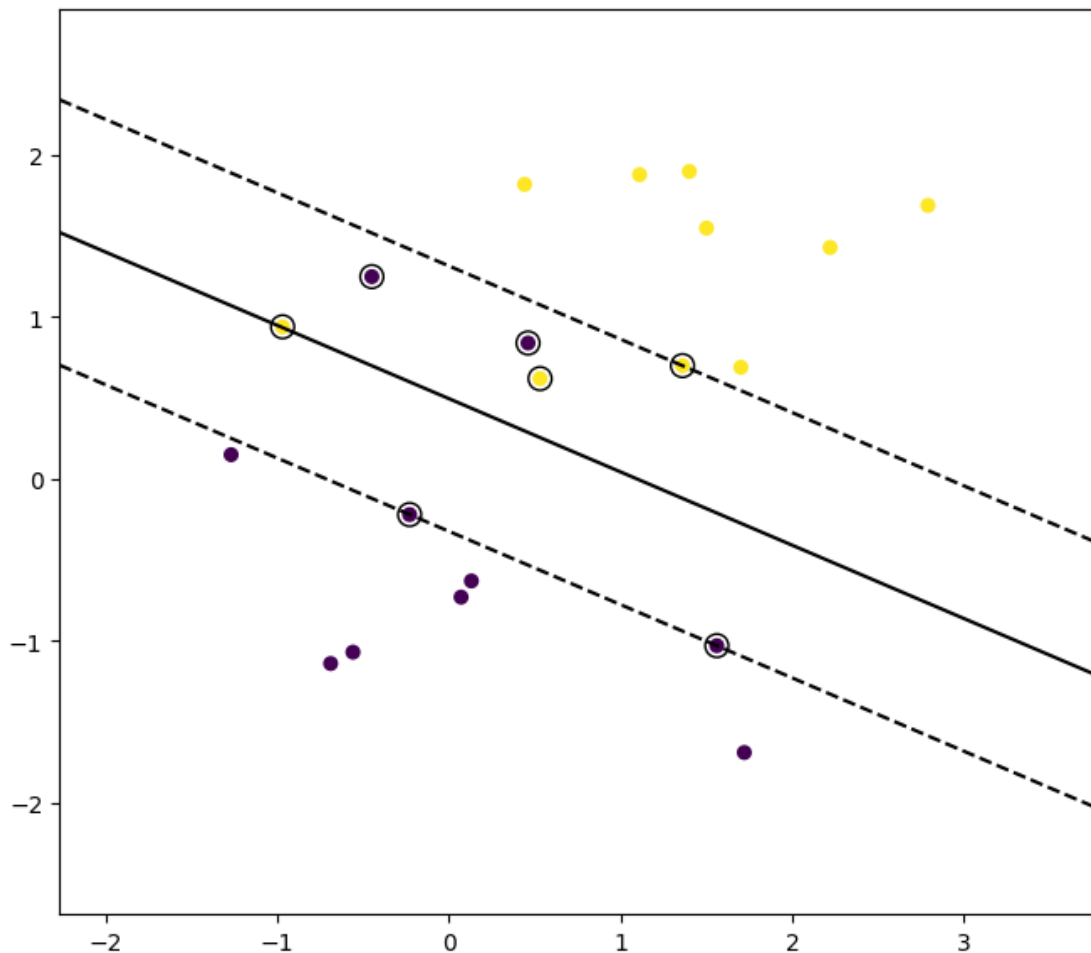


3.2.1 Exercise 3 (CORE)

Fit the SVC model using the **linear kernel** and **same choices of $C=100$ and $C=1$** . How have the results changed? Comment on changes related to the margin and the number of support vectors for each class and their location relative to the margin and hyperplane.

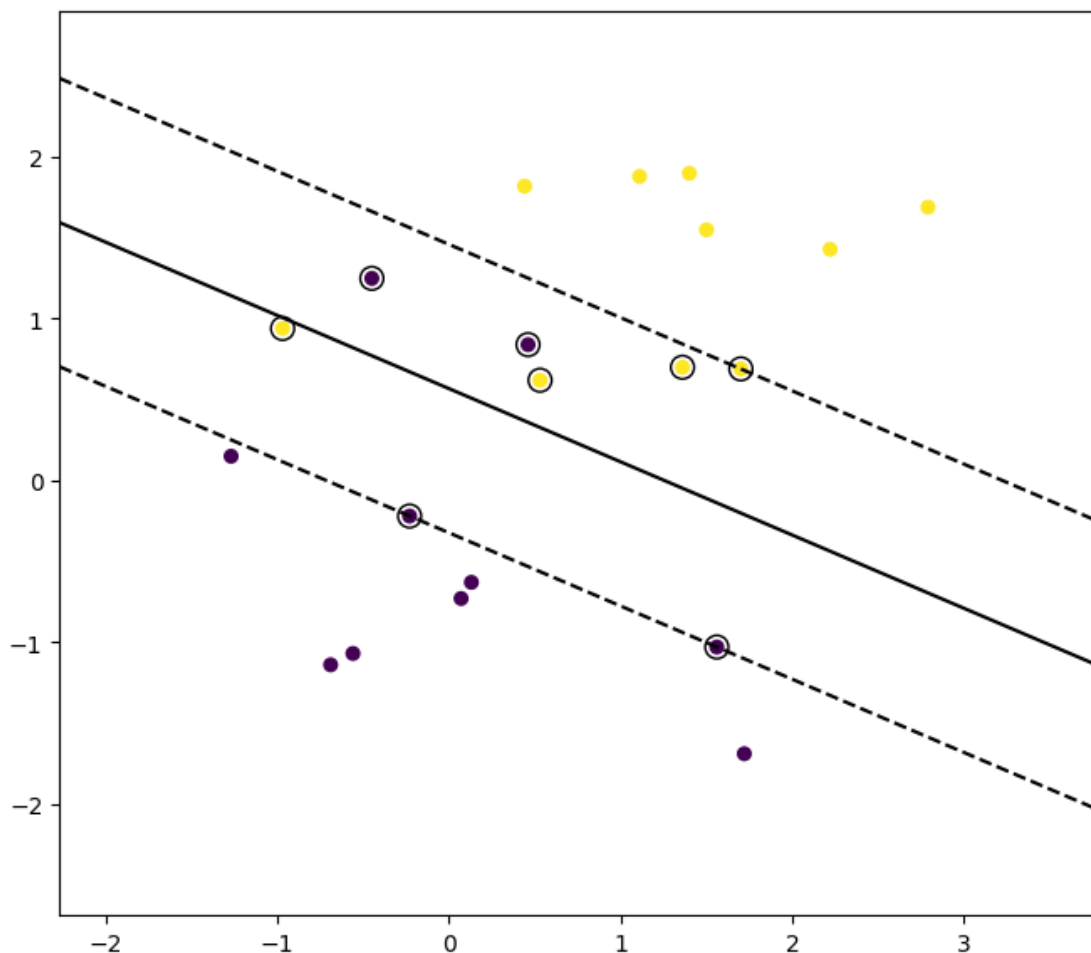

```
[9]: # Fit the SVC on the new data with a larger value of C
svm = SVC(kernel='linear', C=100).fit(X_ex2, y_ex2)

# Visualize the decision boundary, margin and support vectors
plot_margin(svm, X_ex2, y_ex2)
```



```
[10]: # Fit the SVC on the new data with a smaller value of C
svm2 = SVC(kernel='linear', C=1).fit(X_ex2, y_ex2)

# Visualize the decision boundary, margin and support vectors
plot_margin(svm2, X_ex2, y_ex2)
```



Now, the data are no longer linearly separable, i.e. there is no line that can perfectly separate these two classes. Comparing the results with the previous case, we observe:

- the margins are now larger for the same choices of C
- for $C=100$, the number of support vectors has increased from 3 to 7 (three yellow and four purple). Before all points were on the correct side of the margin, but now in non-separable case, there are two purple points on the wrong side of the hyperplane, and two yellow points on the correct side of the hyperplane but on the wrong side of the margin.
- for $C=1$, the number of support vectors has increased to 8. The behavior of the support vectors has changed for the yellow class, with one additional support vector and one point now on the wrong side of the margin.

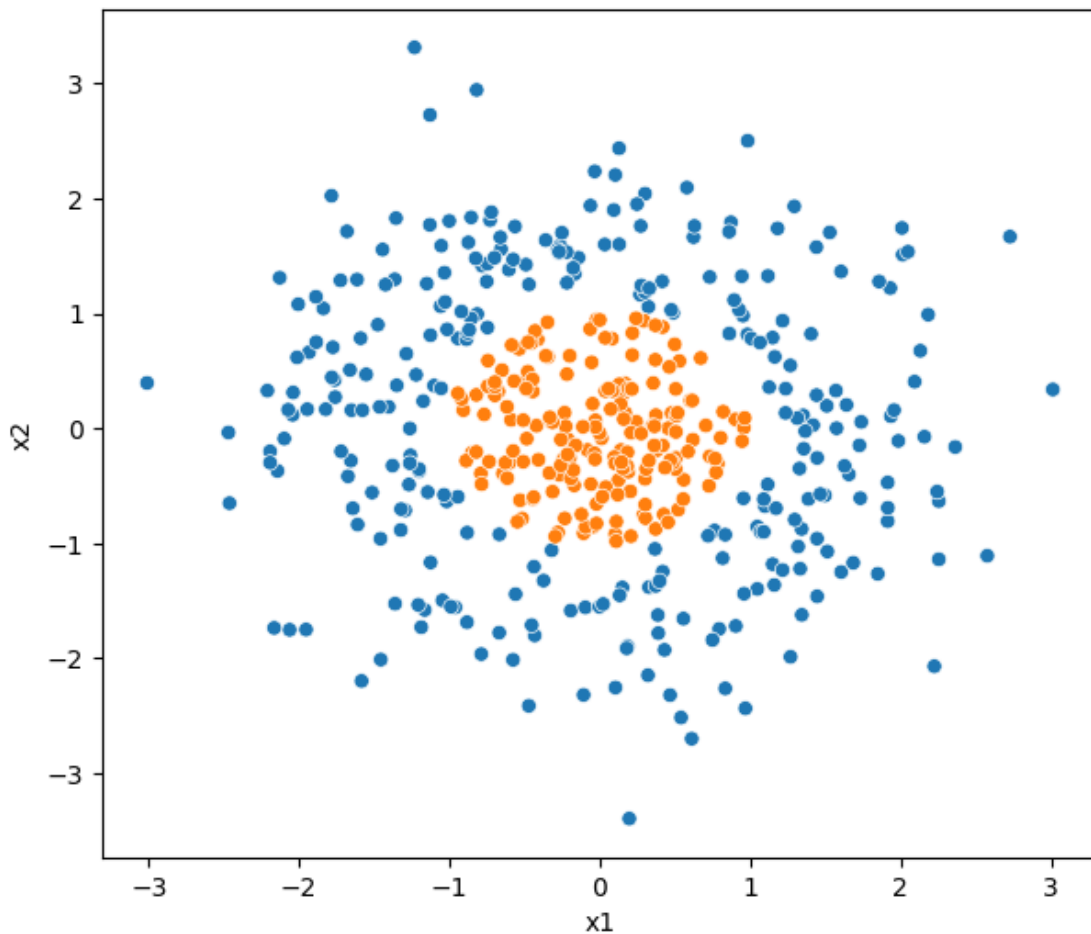
3.3 Nonlinear Separation

Next, we will look at a new data set that is not linearly separable, but can be perfectly separated by a nonlinear decision boundary. We will start by loading and visualizing the data.

```
[11]: # Load the third data set
ex3 = pd.read_csv("ex3.csv")

# Visualize the data
plt.figure(figsize=(7,6))
sns.scatterplot(x='x1', y='x2', hue='y', data=ex3, legend=False)
plt.show()

# Extracting the features and output and encoding y
X_ex3 = np.array(ex3.drop('y', axis=1))
y_ex3 = LabelEncoder().fit_transform(ex3.y)
```

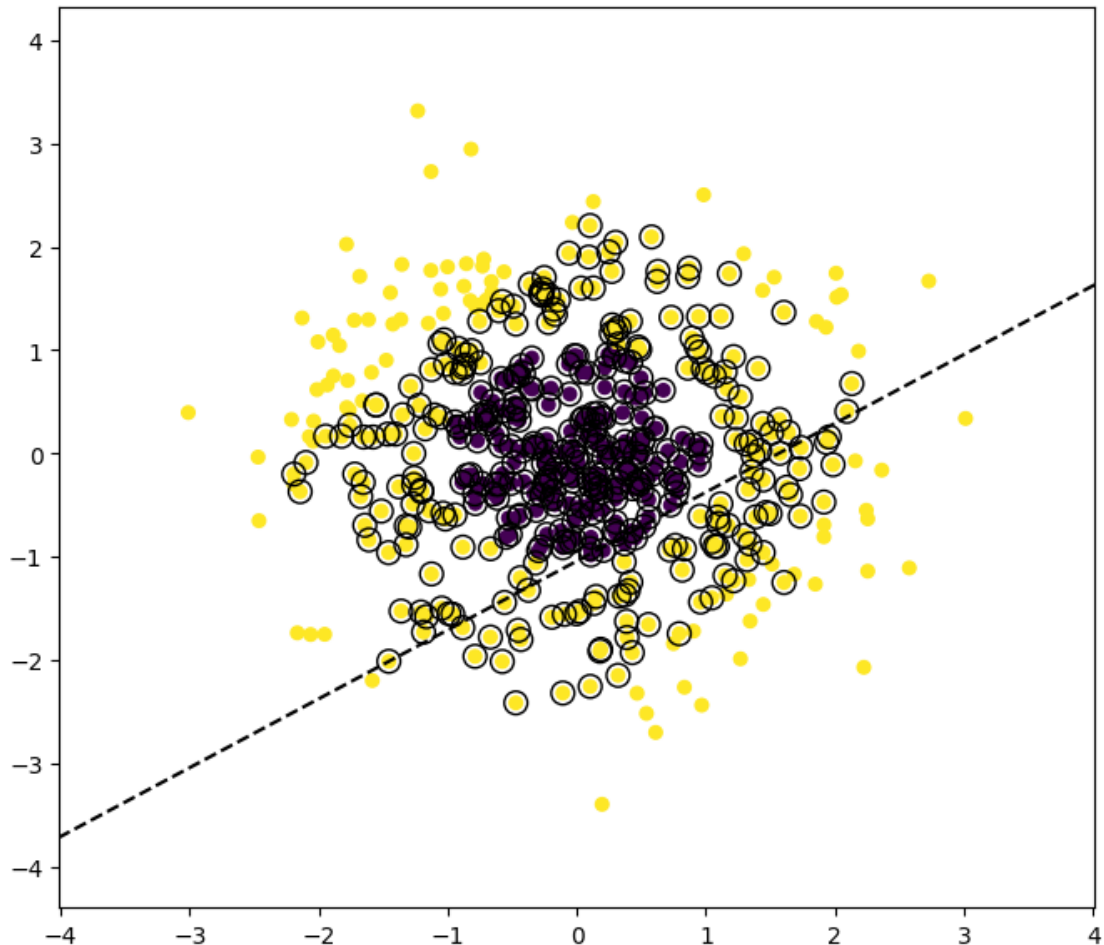


Let's start by trying to fit a linear SVC. As expected, the results are terrible and the number of support vectors is very large.

```
[12]: # Fit the SVC with a linear kernel on the new data
svm_lin = SVC(kernel='linear', C=100).fit(X_ex3, y_ex3)
```

```
# Visualize the decision boundary, margin and support vectors
plot_margin(svm_lin, X_ex3, y_ex3)

print('Number of support vectors for the linear model:', svm_lin.n_support_)
```



Number of support vectors for the linear model: [185 198]

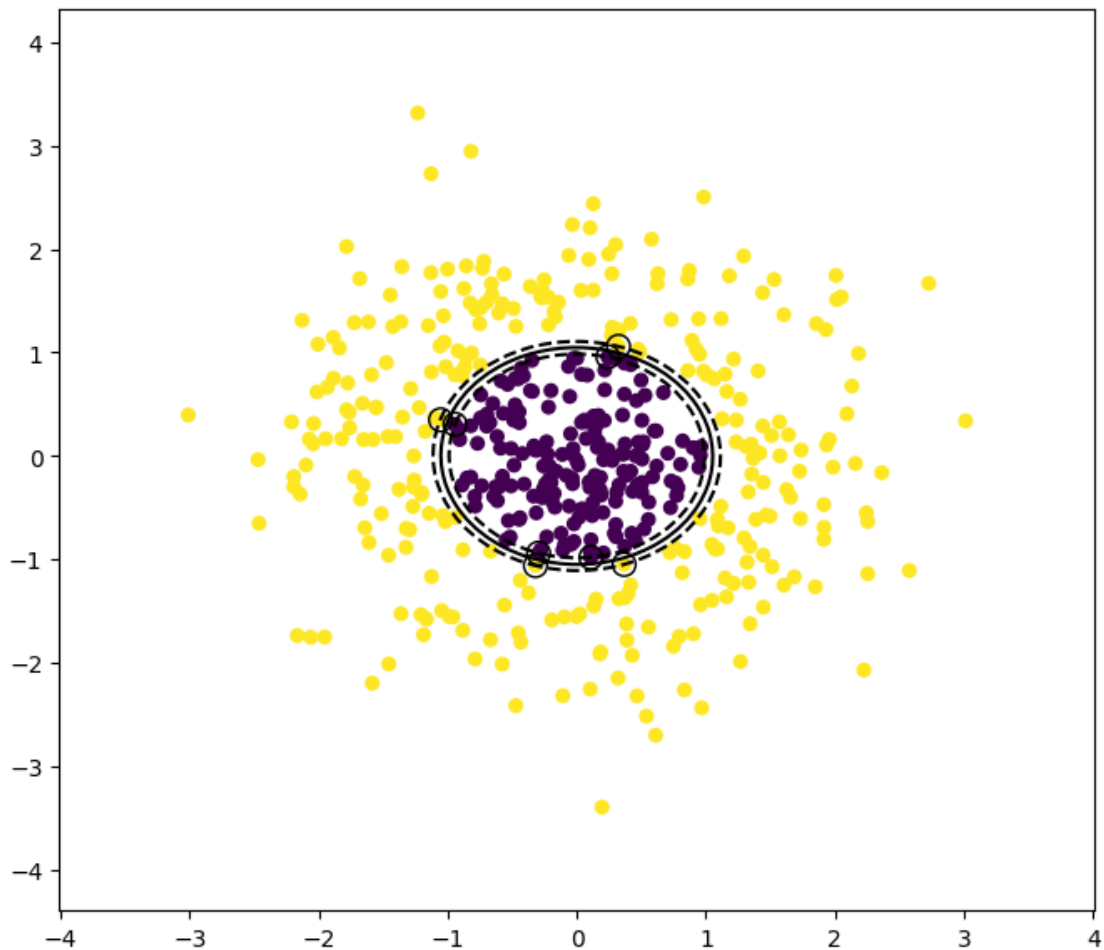
3.3.1 Exercise 4 (CORE)

- Fit an SVM with a **polynomial kernel with degree 2** and $C=100$.
- Visualize the margin and decision boundary and comment on the number of support vectors compared to the linear model.
- Compute and visualize the confusion matrix using `ConfusionMatrixDisplay.from_estimator`. Is the fitted SVM able to perfectly separate the classes?

```
[13]: # Fit the SVM
svm = SVC(kernel = 'poly', degree = 2, C = 100).fit(X_ex3, y_ex3)

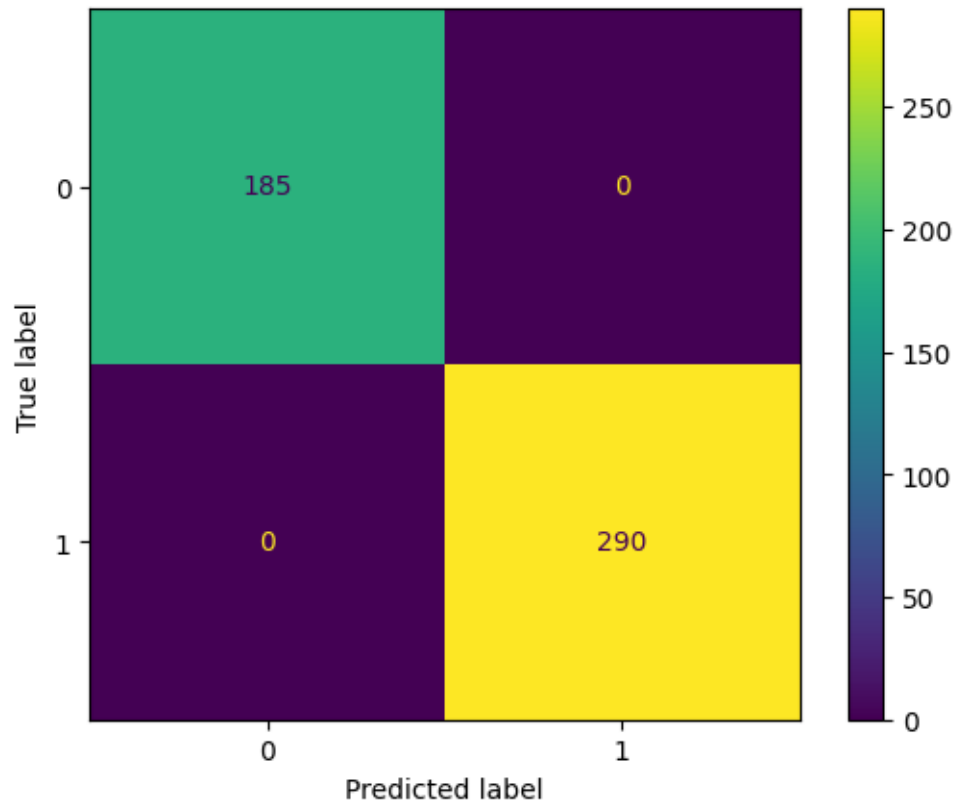
# Visualize the decision boundary, margin and support vectors
plot_margin(svm, X_ex3, y_ex3)

print('Number of support vectors for the nonlinear model:', svm.n_support_)
```



Number of support vectors for the nonlinear model: [4 4]

```
[14]: ConfusionMatrixDisplay.from_estimator(svm, X_ex3, y_ex3)
plt.show()
```



- The polynomial kernel with degree two produces a circular decision boundary, which fits the data well and is able to perfectly separate the two classes.
- There are now only eight support vectors, which is much reduced compared to the linear model.

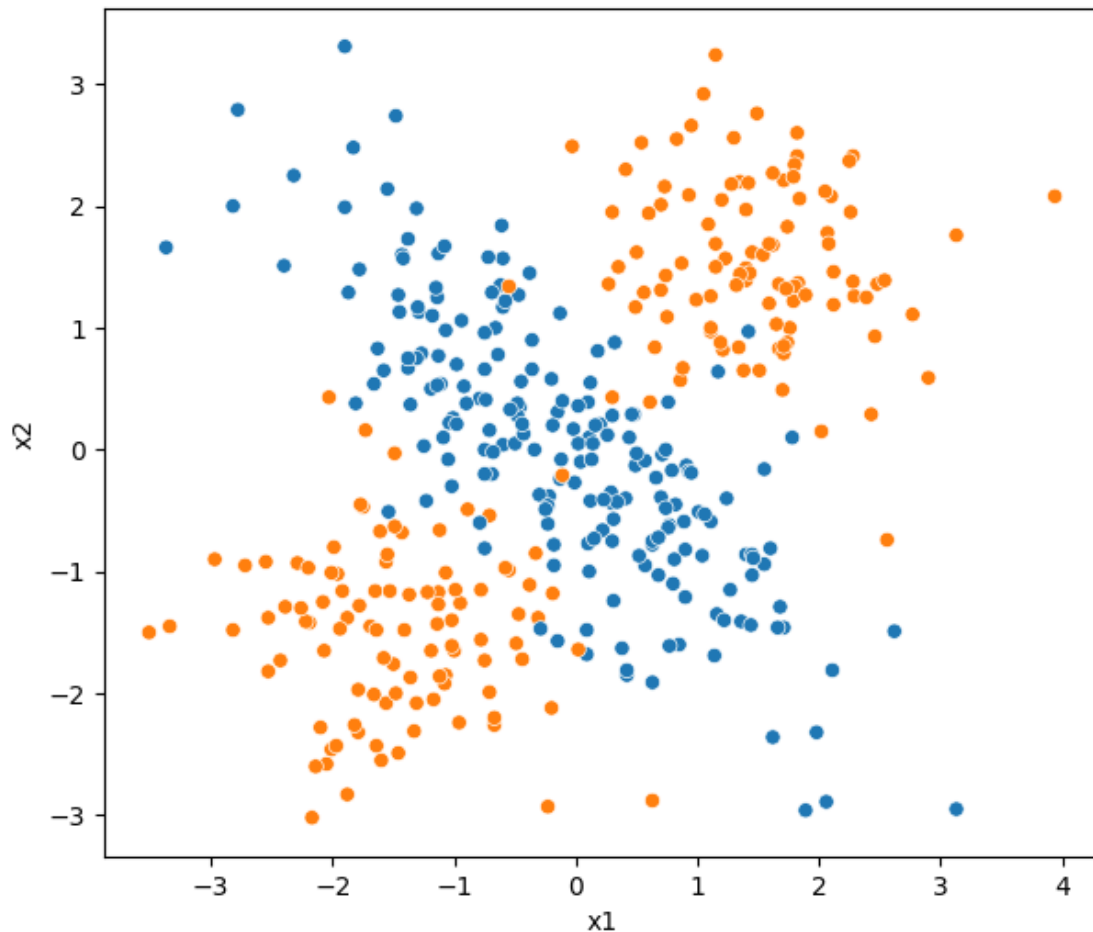
3.4 Kernels for SVMs and Parameter-Tuning

Next, we will consider an more complicated data, and explore using grid search to tune the model parameters and the effect of different kernels.

```
[15]: # Load the fourth data set
ex4 = pd.read_csv("ex4.csv")

# Visualize the data
plt.figure(figsize=(7,6))
sns.scatterplot(x='x1', y='x2', hue='y', data=ex4, legend=False)
plt.show()

# Extracting the features and output and encoding y
X_ex4 = np.array(ex4.drop('y', axis=1))
y_ex4 = LabelEncoder().fit_transform(ex4.y)
```



Let's start with the polynomial kernel that we saw in the previous exercise and use cross-validation to tune both the degree of polynomial and the penalty parameter.

```
[16]: # SVM with polynomial kernel
svm = SVC(kernel='poly', coef0=1)

# Grid search over C and the degree of the polynomial
degrees = [1,2,3,4]
C = np.linspace(0.1, 10, 100)
cv = GridSearchCV(
    svm,
    param_grid = {'C': C,
                  'degree': degrees},
    cv = KFold(5, shuffle = True, random_state = 0)
)

# Fit and tune the model
```

```
cv.fit(X_ex4, y_ex4)
```

```
# Get the best model parameters and the accuracy of the model
```

```
print("Params: ", cv.best_params_)
```

```
print("Avg Accuracy: ", cv.best_score_)
```

```
Params: {'C': 3.1, 'degree': 2}
```

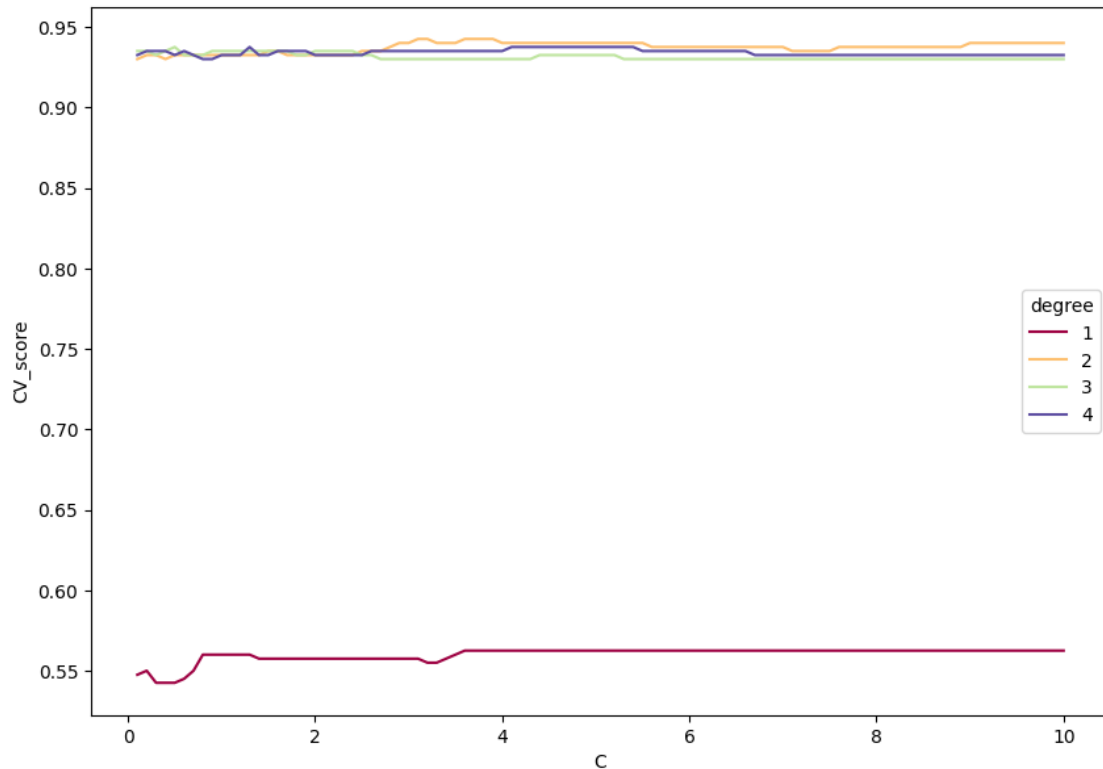
```
Avg Accuracy: 0.9425000000000001
```

3.4.1 Exercise 5 (CORE)

- Run the following code to plot the CV accuracy. Based on this plot, would you use the best parameter values printed above or choose different values? Why?
- Try changing the random state in Kfold to 11205 (or any other number of your choice). Do the best parameter values change? If so, why do you think this happens?
- For your selected parameters, fit the svm and plot the decision boundary and margin.

```
[17]: # Store cv scores in a data frame
cv_accuracy = pd.DataFrame(cv.cv_results_
                           ).filter(['param_C',
                                     ↪ 'param_degree', 'mean_test_score'])
                           .rename(columns={'param_C': 'C',
                                     ↪ 'param_degree': 'degree', 'mean_test_score': 'CV_score'})

# Plot the CV scores
plt.figure(figsize=(10,7))
sns.lineplot(x='C', y='CV_score', data = cv_accuracy, hue = 'degree',
             ↪ palette="Spectral")
plt.show()
```

We observe a major improvement with any degree above one, which is expected, as a linear decision boundary is clearly not sufficient for this data. The curves for degrees of 2,3,4 are quite similar, thus we would prefer the simpler model with a degree of 2 (which is also the degree found by grid search). In addition, the CV scores are quite flat as a function of the penalty parameter, thus, we would prefer the simpler model with a stronger penalty (smaller C). Based on these observations, the best parameter values found by the grid search are reasonable.

Next, let's try changing the random state.

```
[18]: cv2 = GridSearchCV(
    svm,
    param_grid = {'C': C,
                  'degree': degrees},
    cv = KFold(5, shuffle = True, random_state = 11205)
)

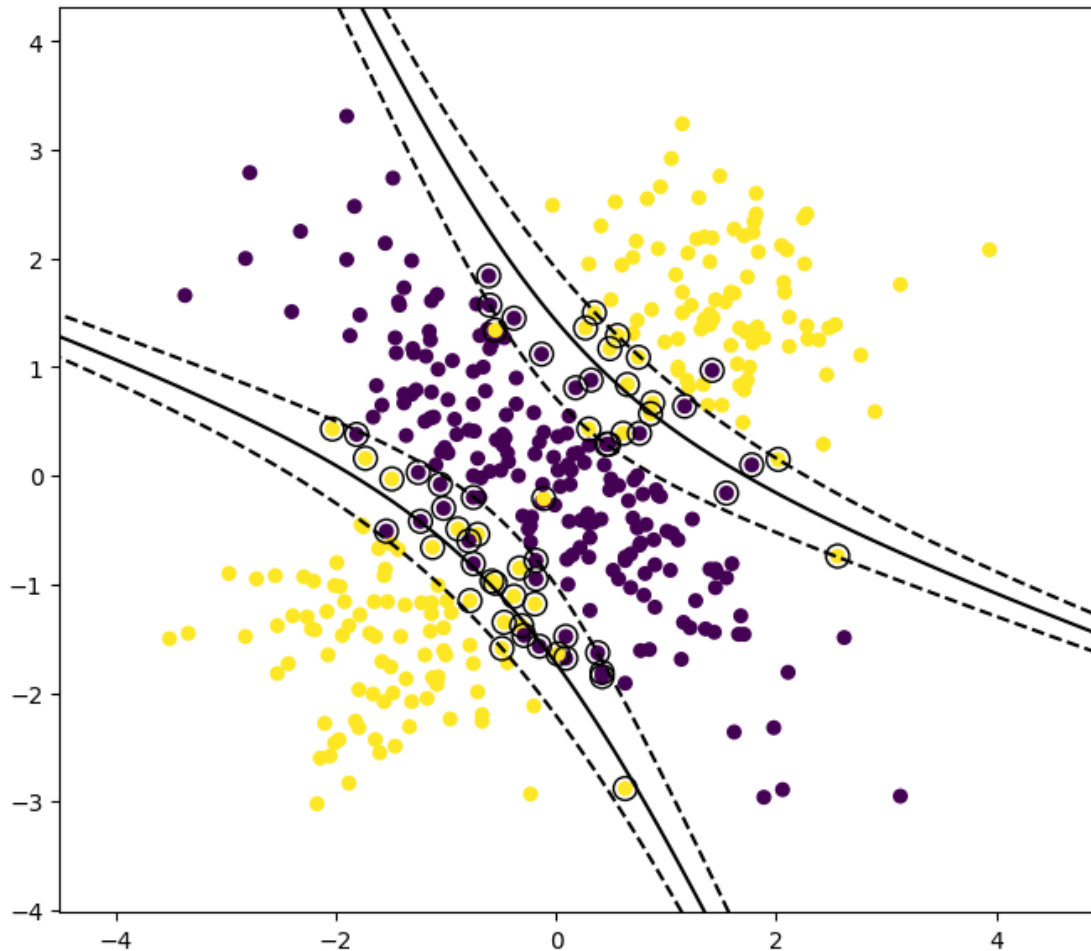
# Fit and tune the model
cv2.fit(X_ex4, y_ex4)

# Get the best model parameters and the accuracy of the model
print("Params: ", cv2.best_params_)
print("Avg Accuracy: ", cv2.best_score_)
```

```
Params: {'C': 0.4, 'degree': 3}
Avg Accuracy: 0.9325000000000001
```

Now, the best degree is 3 and penalty parameter is smaller $C=0.4$. Because the curves are so flat as a function of C and very similar across degrees, the minimum is not well defined. Thus, we find that the best parameters returned are quite variable, when changing how we randomly split the data.

```
[19]: # Visualize the decision boundary
plot_margin(cv.best_estimator_, X_ex4,y_ex4)
```



3.4.2 Exercise 6 (CORE)

- Repeat the grid search above with a polynomial kernel but set the coefficient term `coef0=0` (this is the default value). Redraw the plot of the CV accuracy. How have the results changed? Can you explain why?

```
[20]: # SVM with polynomial kernel
svm0 = SVC(kernel='poly', coef0=0)

# Grid search over C and the degree of the polynomial
degrees = [1,2,3,4]
C = np.linspace(0.1, 10, 100)
cv0 = GridSearchCV(
    svm0,
    param_grid = {'C': C,
                  'degree': degrees},
    cv = KFold(5, shuffle = True, random_state = 0)
)

# Fit and tune the model
cv0.fit(X_ex4, y_ex4)

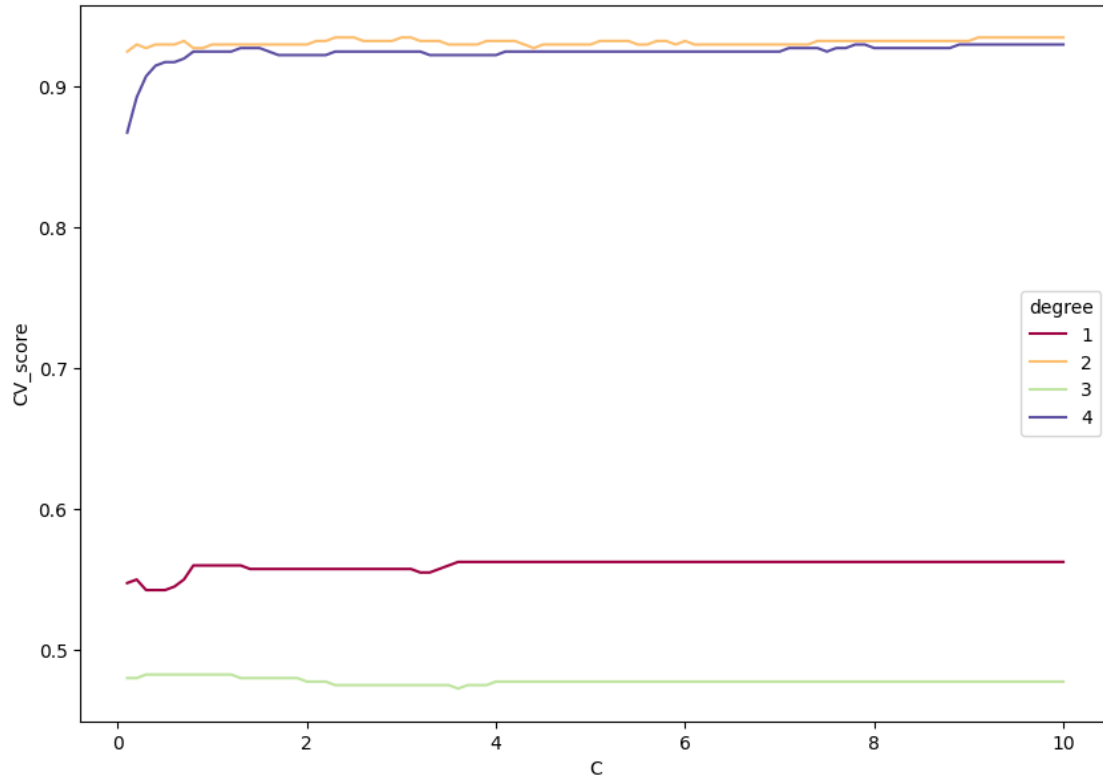
# Get the best model parameters and the accuracy of the model
print("Params: ", cv0.best_params_)
print("Avg Accuracy: ", cv0.best_score_)

# Store cv scores in a data frame
cv_accuracy = pd.DataFrame(cv0.cv_results_
                           ).filter(['param_C',
                                     'param_degree', 'mean_test_score'])
cv_accuracy = cv_accuracy.rename(columns={'param_C': 'C',
                                     'param_degree': 'degree', 'mean_test_score': 'CV_score'})

# Plot the CV scores
plt.figure(figsize=(10,7))
sns.lineplot(x='C', y='CV_score', data = cv_accuracy, hue = 'degree',
             palette="Spectral")
plt.show()
```

Params: {'C': 2.3000000000000003, 'degree': 2}

Avg Accuracy: 0.9349999999999999



The main difference is that degree three now performs quite terribly. Remember that the kernel trick implicitly maps the features to a new (higher-dimensional) feature space. When the coefficient is set to zero, this new feature space has a lower dimension, compared with a coefficient of 1; thus the model is less flexible. To see why, let's focus on the one-dimensional setting:

$$k(x_n, x_{n'}) = (x_n x_{n'})^2 = x_n^2 x_{n'}^2 = \phi(x_n)^T \phi(x_{n'}),$$

where $\phi(x) = x^2$, i.e. we only have one new feature. Instead, with a coefficient of 1, we have

$$k(x_n, x_{n'}) = (x_n x_{n'} + 1)^2 = x_n^2 x_{n'}^2 + 2x_n x_{n'} + 1 = \phi(x_n)^T \phi(x_{n'}),$$

where $\phi(x) = (1, \sqrt{2}x, x^2)$, i.e. we have three new features.

3.4.3 Exercise 7 (CORE)

Consider instead the RBF kernel.

- Use grid search to tune both the penalty parameter C and the inverse bandwidth parameter γ .
- Plot the decision boundary. How do the results compare to the SVM with a polynomial kernel?

```
[21]: # SVM with rbfkernel
      svm2 = SVC(kernel='rbf')
```

```

# Grid search over C and the inverse bandwidth
gamma = [1/4,1/2,1,2,4]
C = np.linspace(0.1, 10, 100)
cv2 = GridSearchCV(
    svm2,
    param_grid = {'C': C,
                  'gamma': gamma},
    cv = KFold(5, shuffle = True, random_state = 0)
)

# Fit and tune the model
cv2.fit(X_ex4, y_ex4)

# Get the best model parameters and the accuracy of the model
print("Params: ", cv2.best_params_)
print("Avg Accuracy: ", cv2.best_score_)

# Store cv scores in a data frame
cv_accuracy = pd.DataFrame(cv2.cv_results_
                           ).filter(['param_C', 'param_gamma', 'mean_test_score']
                           ).rename(columns={'param_C': 'C',
                                             'param_gamma': 'gamma', 'mean_test_score': 'CV_score'})

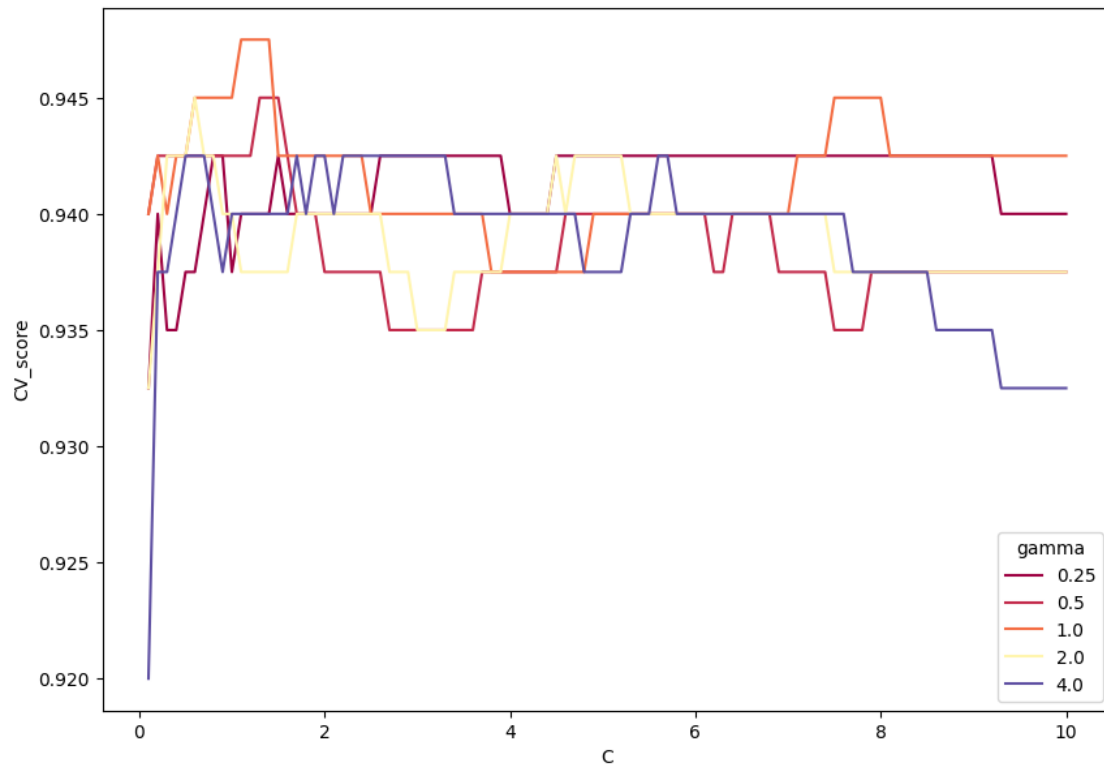
# Plot the CV scores
plt.figure(figsize=(10,7))
sns.lineplot(x='C', y='CV_score', data = cv_accuracy, hue = 'gamma',
             palette="Spectral")
plt.show()

```

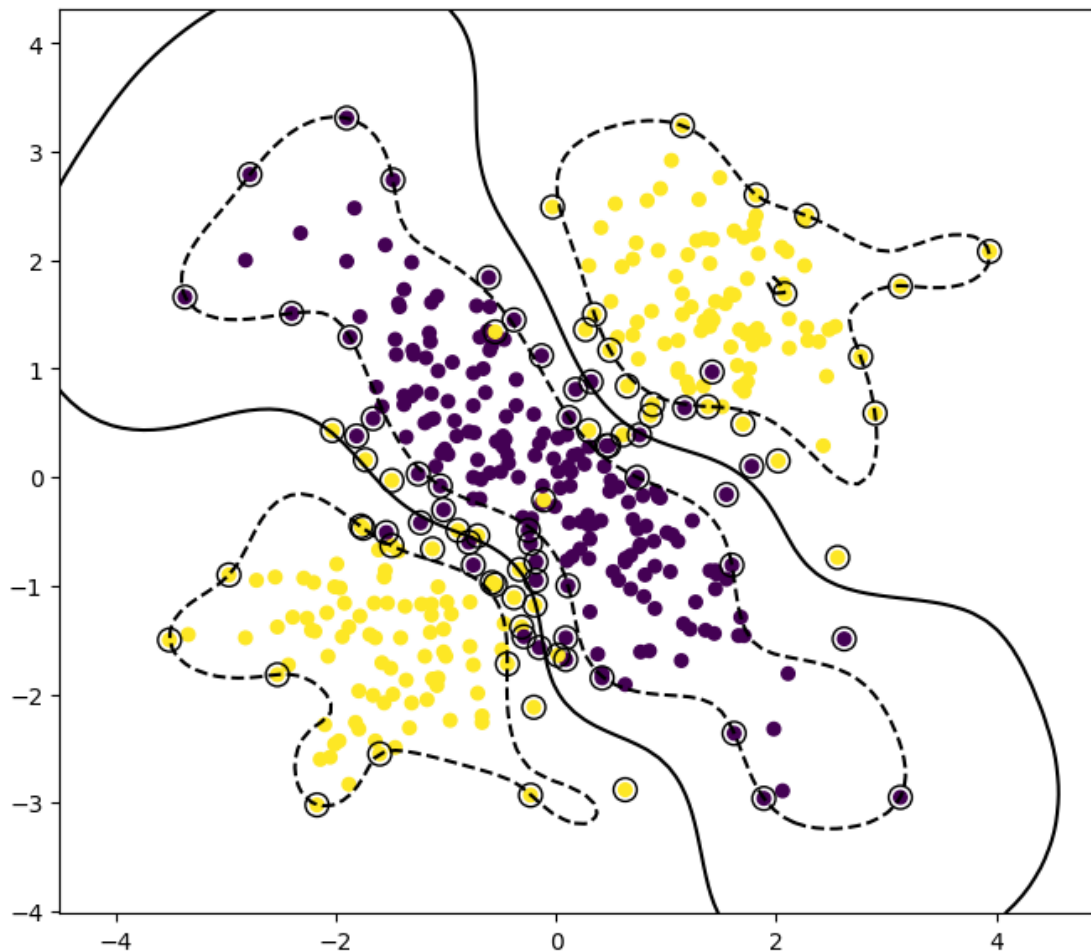
```

Params:  {'C': 1.1, 'gamma': 1}
Avg Accuracy:  0.9475

```



```
[22]: # Visualize the decision boundary
      plot_margin(cv2.best_estimator_, X_ex4,y_ex4)
```



```
[23]: print('Number of support vectors with polynomial kernel:', cv.best_estimator_.
      ↪ n_support_)
      print('Number of support vectors with RBF kernel:', cv2.best_estimator_.
      ↪ n_support_)

      print('CV accuracy with polynomial kernel:', cv.best_score_)
      print('CV accuracy with RBF kernel:', cv2.best_score_)
```

Number of support vectors with polynomial kernel: [31 31]

Number of support vectors with RBF kernel: [44 48]

CV accuracy with polynomial kernel: 0.9425000000000001

CV accuracy with RBF kernel: 0.9475

For the RBF kernel, the CV results seems fairly robust to the choice of bandwidth and penalty parameter. As discussed in lectures, a small length scale (larger γ) tends to lead to very wiggly functions and overfitting. The GridSearchCV selects the value of $\gamma = 1$, which does produces a slightly wiggly boundary, and possibly we want to use a slightly smaller value.

Comparing the CV score, the results with the RBF kernel are slightly better than with the polynomial kernel. However, the results are not much different.

3.5 Model Assessment

As, we learned last week, there are many metrics to consider beyond accuracy. For example, below we compute and print the classification report, summarizing the results for precision, recall, f1-score, and accuracy.

```
[24]: from sklearn.metrics import classification_report

# Classification report for the SVM with polynomial kernel
print(classification_report(y_ex4, cv.best_estimator_.predict(X_ex4)))

# Classification report for the SVM with RBF kernel
print(classification_report(y_ex4, cv2.best_estimator_.predict(X_ex4)))
```

	precision	recall	f1-score	support
0	0.92	0.97	0.94	200
1	0.97	0.92	0.94	200
accuracy			0.94	400
macro avg	0.94	0.94	0.94	400
weighted avg	0.94	0.94	0.94	400

	precision	recall	f1-score	support
0	0.95	0.95	0.95	200
1	0.95	0.94	0.95	200
accuracy			0.95	400
macro avg	0.95	0.95	0.95	400
weighted avg	0.95	0.95	0.95	400

We also learned about other tools, such as the ROC curve, AUC, and precision-recall curve. However, since the SVMs are not model-based, **we only obtain hard label assignments when doing predictions**. To overcome this, one heuristic that can be used is **Platt scaling** to convert the SVM output to probabilities. However, these probabilities may not be well calibrated and may be inconsistent with the hard labels.

In the code below, we visualize the probabilities of class assignments across a grid of possible inputs for the tuned SVM polynomial model.

```
[25]: # Fit the polynomial model with optimal prarmeters and the option to compute_
      ↪ the probabilities
svm_poly = SVC(kernel='poly', C=cv.best_params_["C"], degree=cv.
      ↪ best_params_["degree"], probability=True).fit(X_ex4, y_ex4)
```



```

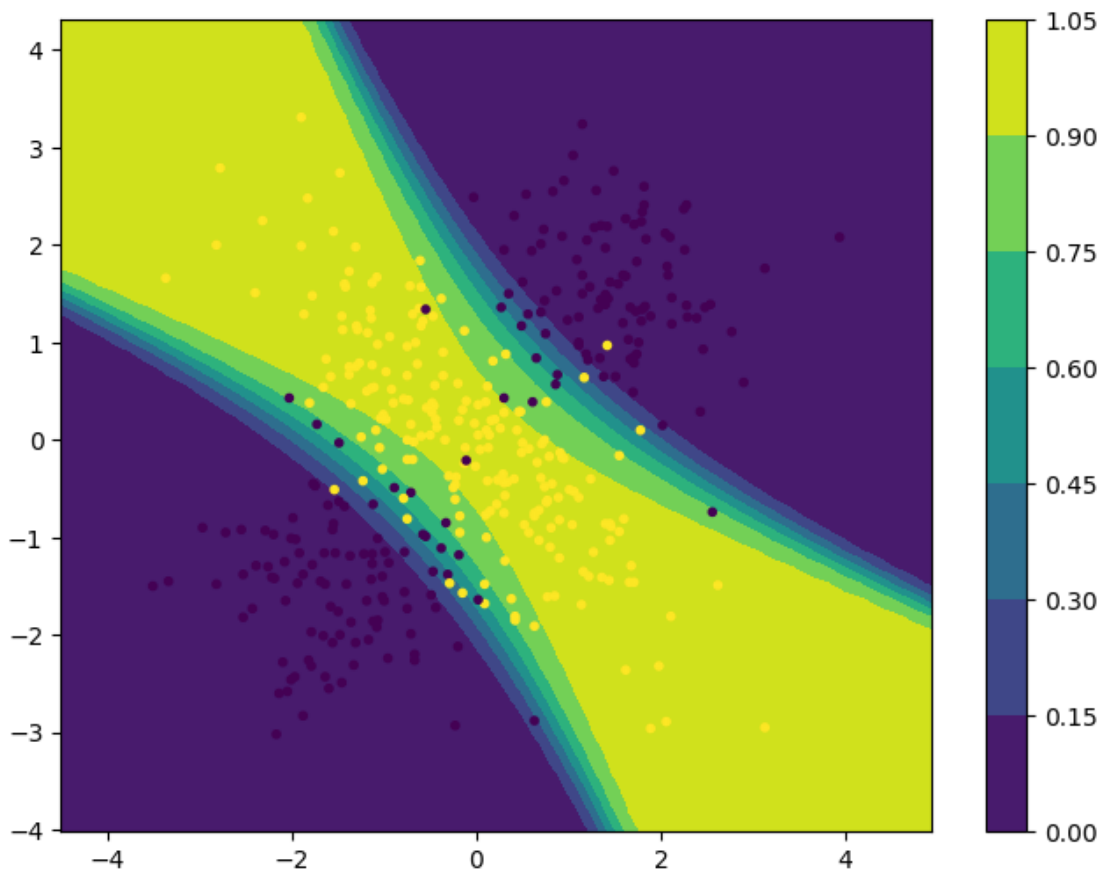
# Create a grid of inputs for plotting
x1lim = [X_ex4[:,0].min(),X_ex4[:,0].max()]
x2lim = [X_ex4[:,1].min(),X_ex4[:,1].max()]

xx1 = np.linspace(x1lim[0]-1, x1lim[1]+1, 50)
xx2 = np.linspace(x2lim[0]-1, x2lim[1]+1, 50)
XX2, XX1 = np.meshgrid(xx2, xx1)

# Calculate the probabiltiy for each point in the grid
xx = np.c_[XX1.ravel(), XX2.ravel()]
xx.shape
P = svm_poly.predict_proba(xx)[: ,0].reshape(XX1.shape)

plt.figure(figsize=(8,6))
plt.contourf(XX1, XX2, P)
plt.colorbar()
plt.scatter(x=X_ex4[:,0], y=X_ex4[:,1],c = 1-y_ex4, marker = '.')
plt.show()

```



Notice how the model is quite **confident in the predictions (probabilities close to 0 or 1)** in the corners of the input space, even where we don't have any data. In some cases, this may be undesirable, as we may not want to make such confident assessments in areas where we have little to no data.

3.5.1 Exercise 8 (EXTRA)

- For the SVM with RBF kernel and optimal CV parameters, repeat the plot above to visualize the probabilities of class assignments.
- How do the results compare with the polynomial kernel? Does this impact your choice of kernel?

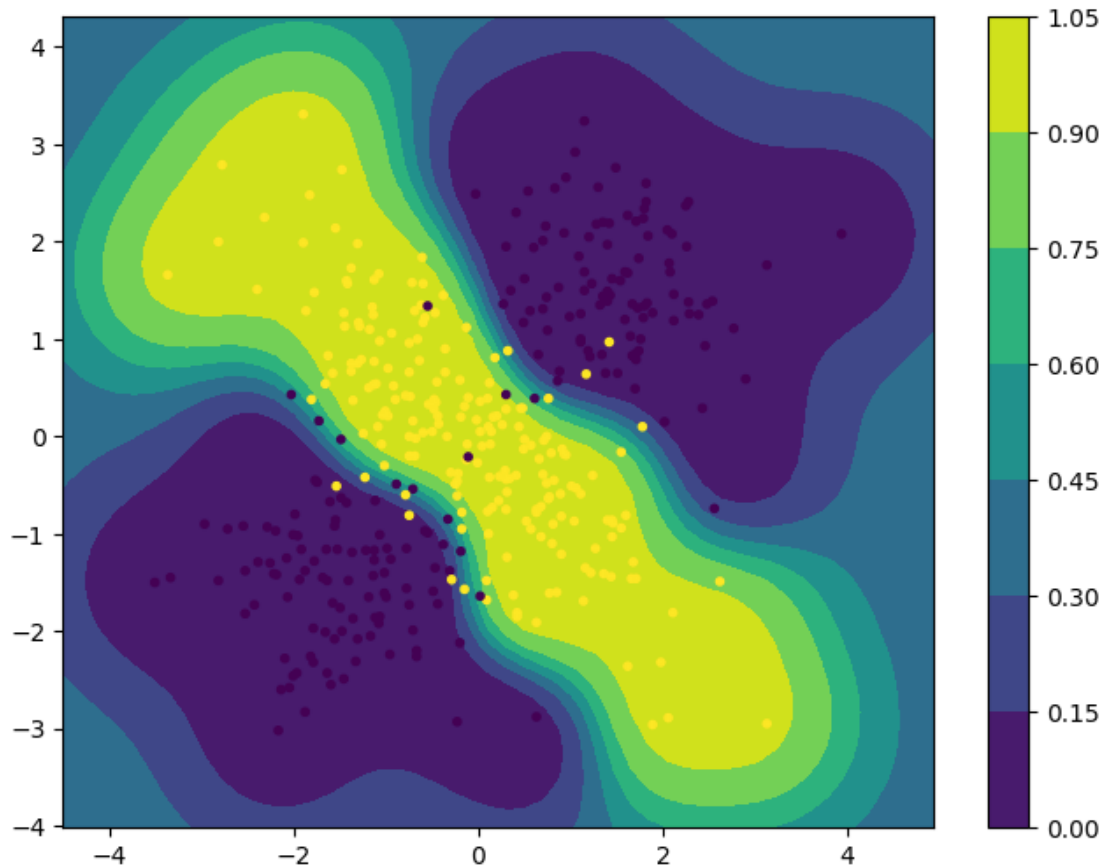
```
[26]: # Fit the rbf model with optimal prarmeters and the option to compute the
      ↪probabilities
svm_rbf = SVC(kernel='rbf', C=cv2.best_params_["C"], gamma=cv2.
      ↪best_params_["gamma"], probability=True).fit(X_ex4, y_ex4)

# Create a grid of inputs for plotting
x1lim = [X_ex4[:,0].min(),X_ex4[:,0].max()]
x2lim = [X_ex4[:,1].min(),X_ex4[:,1].max()]

xx1 = np.linspace(x1lim[0]-1, x1lim[1]+1, 50)
xx2 = np.linspace(x2lim[0]-1, x2lim[1]+1, 50)
XX2, XX1 = np.meshgrid(xx2, xx1)

# Calculate the probabiltiy for each point in the grid
xx = np.c_[XX1.ravel(), XX2.ravel()]
xx.shape
P = svm_rbf.predict_proba(xx)[:,:].reshape(XX1.shape)

plt.figure(figsize=(8,6))
plt.contourf(XX1, XX2, P)
plt.colorbar()
plt.scatter(x=X_ex4[:,0], y=X_ex4[:,1],c = 1-y_ex4, marker = '.')
plt.show()
```

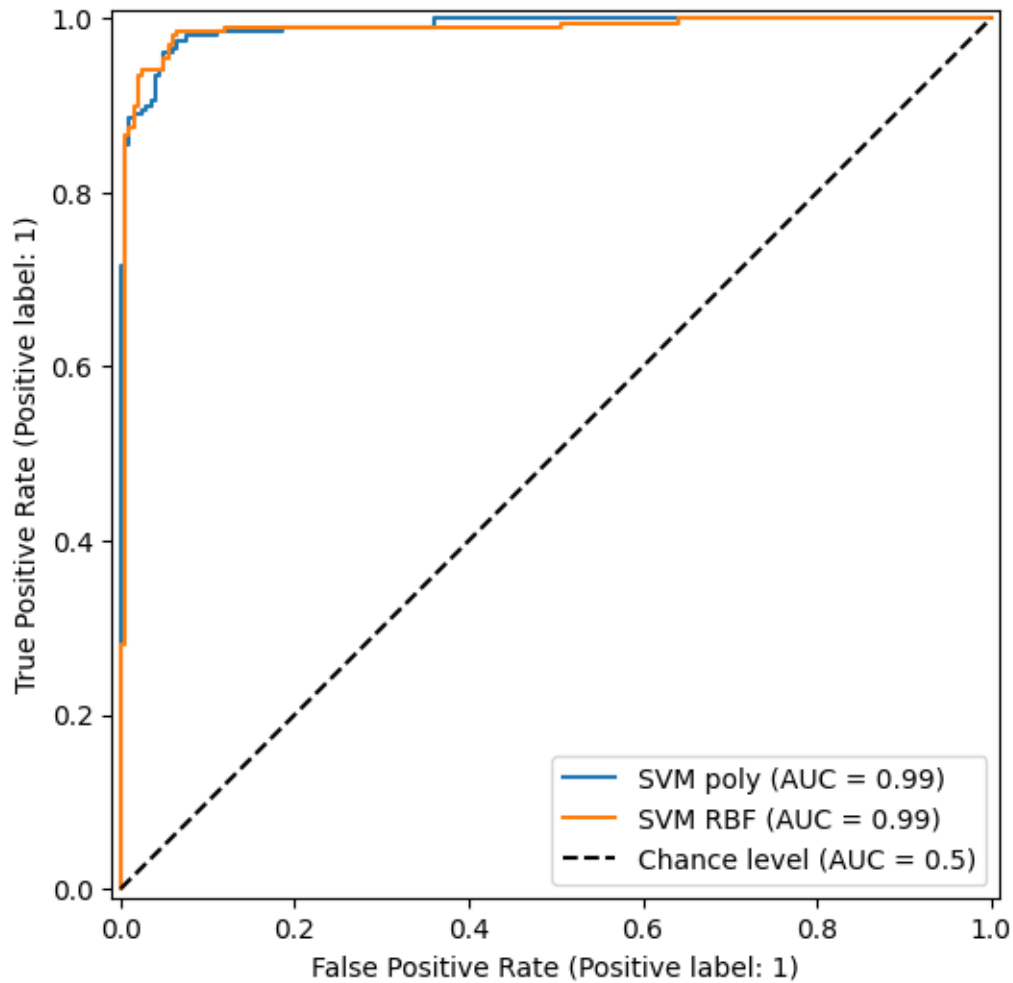


Although the RBF kernel has slightly overwiggly boundaries (which we may improve by choosing a smaller γ), compared with the polynomial kernel, the class probabilities are close to 0.5 in the corners where we have no data. This may be more desirable, to avoid making any strong statements in such regions.

We can also plot the ROC curve for both models from the Platt scaling probabilities, which are very similar for both models.

```
[27]: from sklearn.metrics import RocCurveDisplay

# Plot the ROC curve
fig, ax = plt.subplots(figsize=(6,6))
RocCurveDisplay.from_estimator(svm_poly, X_ex4, y_ex4, ax = ax, name = 'SVM_
    ↳poly')
RocCurveDisplay.from_estimator(svm_rbf, X_ex4, y_ex4, plot_chance_level=True, ax_
    ↳= ax, name = 'SVM RBF')
plt.show()
```



4 Spam Data

Let's consider the spam data available on [kaggle](#). The dataset includes thousands of messages that have been marked as `spam` or `ham`. We will start by loading and exploring the data.

```
[28]: # Load the data
df = pd.read_csv("spam.csv", encoding='latin-1')

# Check information about the dataset
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---

```

```

0    v1          5572 non-null    object
1    v2          5572 non-null    object
2    Unnamed: 2    50 non-null    object
3    Unnamed: 3    12 non-null    object
4    Unnamed: 4     6 non-null    object
dtypes: object(5)
memory usage: 217.8+ KB
None

```

Let's do some basic data cleaning.

```

[29]: # Drop unnecessary columns
columns_to_drop = ["Unnamed: 2", "Unnamed: 3", "Unnamed: 4"]
df.drop(columns=columns_to_drop, inplace=True)

# Rename columns for better readability
new_column_names = {"v1": "Category", "v2": "Message"}
df.rename(columns=new_column_names, inplace=True)

print(df.info())

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Category    5572 non-null    object
1   Message     5572 non-null    object
dtypes: object(2)
memory usage: 87.2+ KB
None

```

Looking at the first few rows of the data (below), we see that we will need to do some feature engineering to covert the text messages into numeric features.

```

[30]: df.head()

```

```

[30]:   Category      Message
0     ham  Go until jurong point, crazy.. Available only ...
1     ham                Ok lar... Joking wif u oni...
2    spam  Free entry in 2 a wkly comp to win FA Cup fina...
3     ham  U dun say so early hor... U c already then say...
4     ham  Nah I don't think he goes to usf, he lives aro...

```

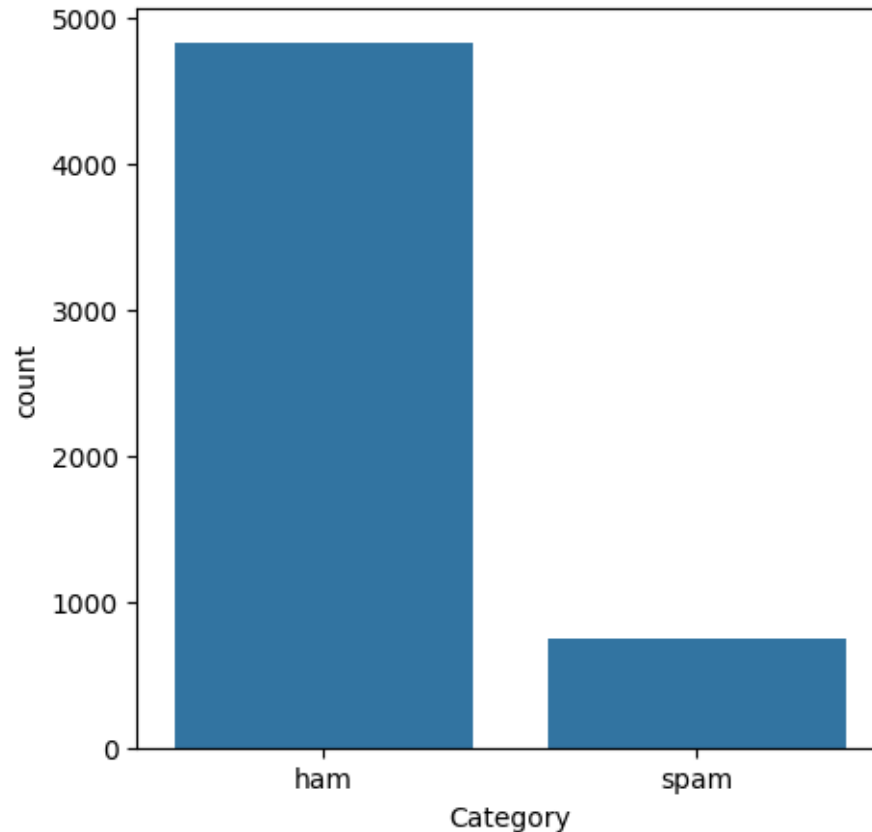
Let's start by splitting our data into features and targets. Notice that the dataset contains, as expected, less spam emails than ham.

```

[31]: # Split the dataset into features (X) and target (y)
X = df["Message"]
y = df["Category"]

```

```
plt.figure(figsize=(5,5))
sns.countplot(x=y)
plt.show()
```



We are going to use a simple **bag of words** transformation to convert our message to numeric features through sklearn's transformer: [CountVectorizer](#).

We will incorporate it into our pipeline later, but you can run the code below to get a sense of how it transforms the messages.

For simplicity, we set: - `binary=True` which returns binary features indicating the presence/absence of the words in the vocabulary - `max_features=50` and `stop_words="english"` which automatically extracts the vocabulary as the 50 most frequent words among the messages, excluding common english words like "and", "the", "him".

We can print the attribute `.vocabulary_` of our fitted transformer to see the words of our vocabulary.

```
[32]: # Create count vectorizer for bag of words feature extraction
from sklearn.feature_extraction.text import CountVectorizer
```

```
fe = CountVectorizer(max_features=50, stop_words="english", binary=True).fit(X)
X_ = fe.transform(X)

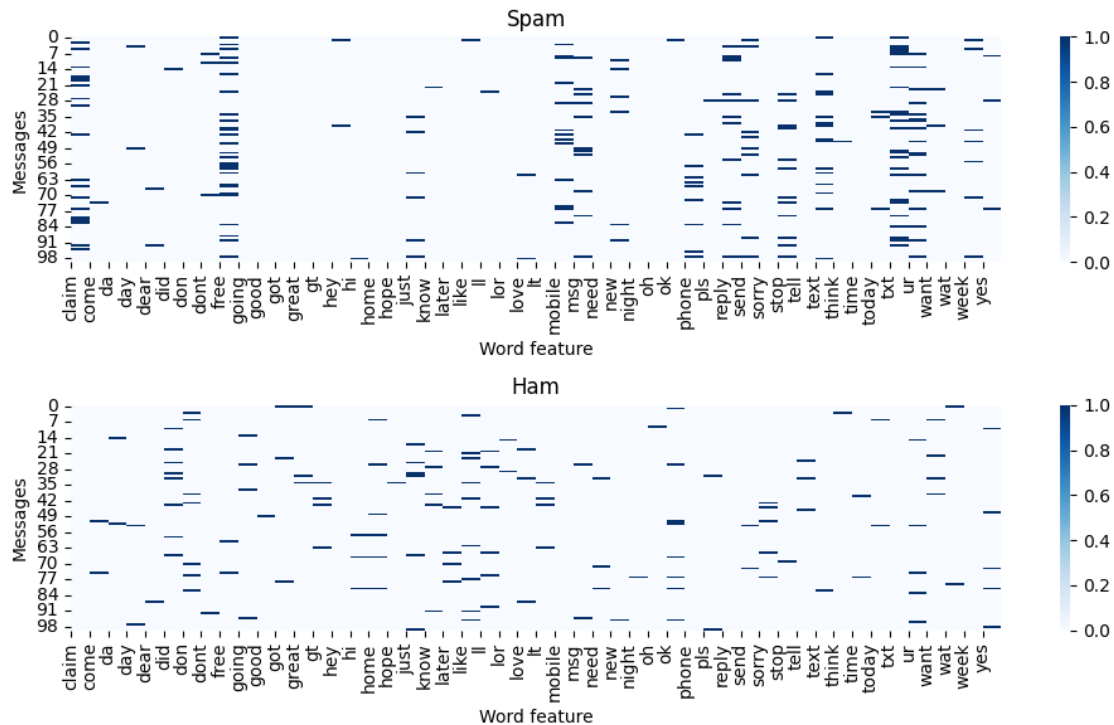
# print vocabulary
fe.vocabulary_
```

```
[32]: {'great': 12,
      'got': 11,
      'wat': 47,
      'ok': 32,
      'free': 8,
      'text': 40,
      'txt': 44,
      'don': 6,
      'think': 41,
      'hey': 14,
      'week': 48,
      'like': 21,
      'send': 36,
      'claim': 0,
      'mobile': 26,
      'home': 16,
      'want': 46,
      'today': 43,
      'day': 3,
      'reply': 35,
      'oh': 31,
      'yes': 49,
      'did': 5,
      'dont': 7,
      'ur': 45,
      'going': 9,
      'da': 2,
      'lor': 23,
      'just': 18,
      'love': 24,
      'll': 22,
      'know': 19,
      'tell': 39,
      'msg': 27,
      'pls': 34,
      'need': 28,
      'hope': 17,
      'lt': 25,
      'gt': 13,
      'time': 42,
      'sorry': 37,
```

```
'new': 29,  
'later': 20,  
'good': 10,  
'come': 1,  
'hi': 15,  
'stop': 38,  
'night': 30,  
'dear': 4,  
'phone': 33}
```

To visualize the transformed data, let's also draw a heatmap of first 100 messages for spam and ham messages.

```
[33]: # Draw a heatmap of the first 100 messages  
feature_names = fe.get_feature_names_out()  
X_pd_spam = pd.DataFrame(  
    X_.toarray()[y=='spam',:][0:100,:],  
    columns=feature_names  
)  
X_pd_ham = pd.DataFrame(  
    X_.toarray()[y=='ham',:][0:100,:],  
    columns=feature_names  
)  
  
fig, ax = plt.subplots(2,1,figsize=(10,6))  
sns.heatmap(X_pd_spam, cmap='Blues', vmin=0, vmax=1, ax=ax[0])  
ax[0].set_xlabel('Word feature')  
ax[0].set_ylabel('Messages')  
ax[0].set_title('Spam')  
ax[0].set_xticks(np.arange(len(feature_names)), labels = feature_names)  
sns.heatmap(X_pd_ham, cmap='Blues', vmin=0, vmax=1, ax=ax[1])  
ax[1].set_xlabel('Word feature')  
ax[1].set_ylabel('Messages')  
ax[1].set_title('Ham')  
ax[1].set_xticks(np.arange(len(feature_names)), labels = feature_names)  
plt.yticks(rotation=0)  
plt.tight_layout()  
plt.show()
```

Next, we split the data into a train and test set. Due to the class imbalance in the data, we use the option `stratify=y` to maintain the class proportion in the test set.

```
[34]: from sklearn.model_selection import train_test_split

y = LabelEncoder().fit_transform(y)

# Stratify split
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True,
    ↪stratify=y,

                                                    test_size = 0.1,
    ↪random_state=1112)
print(X_train.shape)
print('Proportion of spam in training', np.mean(y_train))
print('Proportion of spam in test', np.mean(y_test))
```

```
(5014,)
Proportion of spam in training 0.1340247307538891
Proportion of spam in test 0.13440860215053763
```

4.0.1 Exercise 9 (CORE)

- Run the following code to fit an SVC and tune the penalty parameter `C`.
- Plot the accuracy, recall, precision, and f1 score as function of `C`. What value of `C` would you

choose and why?

```
[35]: from sklearn.model_selection import StratifiedKFold

# SVM with linear kernel
svm_lin = make_pipeline(
    CountVectorizer(max_features=50, stop_words="english", binary=True),
    SVC(kernel='linear')
)

# Grid search over C
C = np.linspace(0.1, 10, 20)
cv_lin = GridSearchCV(
    svm_lin,
    param_grid = {'svc__C': C},
    cv = StratifiedKFold(5, shuffle = True, random_state = 0),
    scoring = ["accuracy", "f1", "recall", "precision"],
    refit='accuracy' #refit based on accuracy
)

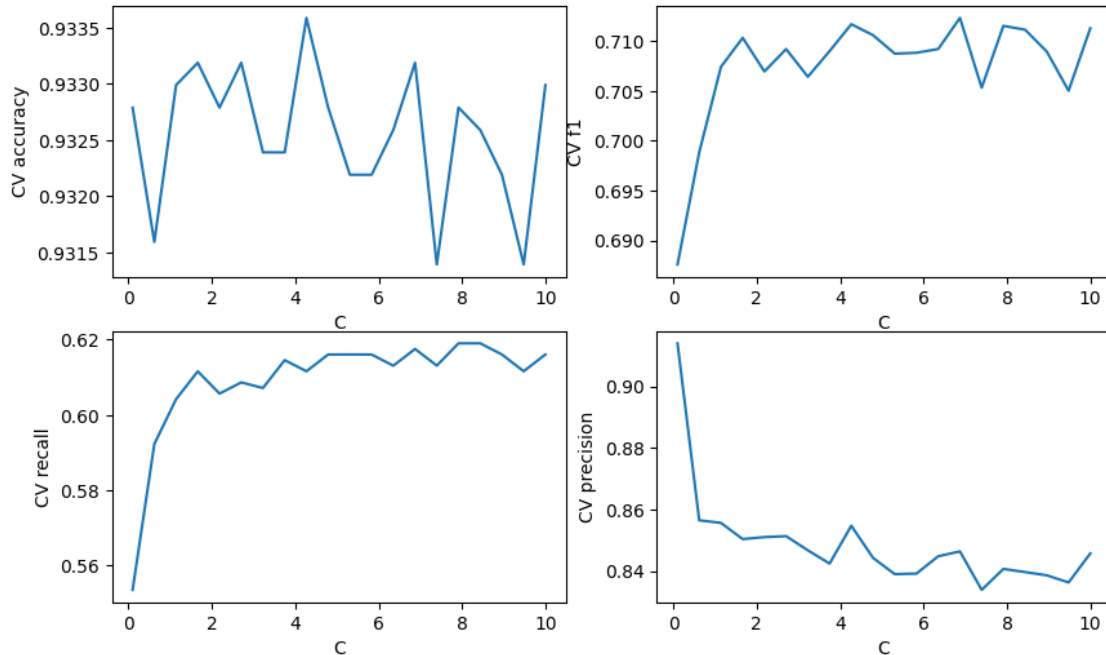
# Fit and tune the model
cv_lin.fit(X_train, y_train)

# Store cv scores in a data frame
cv_accuracy = pd.DataFrame(cv_lin.cv_results_
    ).
    filter(['param_svc__C', 'mean_test_accuracy', 'mean_test_f1',
    'mean_test_recall', 'mean_test_precision'])
    .rename(columns={'param_svc__C':
    'C', 'mean_test_accuracy': 'CV accuracy', 'mean_test_f1': 'CV f1',
    'mean_test_recall': 'CV recall', 'mean_test_precision': 'CV precision'})
```

```
[36]: # Get the best model parameters and the recall of the model
print("Params: ", cv_lin.best_params_)
print("Avg Accuracy: ", cv_lin.best_score_)

# Plot the CV scores
fig, ax = plt.subplots(2,2,figsize=(10,6))
ax = ax.flatten()
sns.lineplot(x='C', y='CV accuracy', data = cv_accuracy, ax=ax[0])
sns.lineplot(x='C', y='CV f1', data = cv_accuracy, ax=ax[1])
sns.lineplot(x='C', y='CV recall', data = cv_accuracy, ax=ax[2])
sns.lineplot(x='C', y='CV precision', data = cv_accuracy, ax=ax[3])
plt.show()
```

```
Params: {'svc__C': 4.268421052631579}
Avg Accuracy: 0.9335884561883212
```



In this setting, if we care more about being precise in classifying emails as spam over recalling all the spam emails. This is because we typically would prefer having a few spam emails show up in our inbox over missing some potentially important emails that are mistakenly sent to the spam folder.

Choosing the smallest value of C gives the highest precision, at the cost of a small drop in accuracy and F1. This may be a reasonable choice depending on how much we prioritize false positives over false negatives.

4.0.2 Exercise 10 (CORE)

- Refit the model with your chosen value of C . You should call your model `svm_lin` or change the name in code for part b.
- Run the code the below to visualize the coefficients and the support vectors for each class. Briefly comment on words/features that are relevant for distinguishing between the classes.

```
[37]: # Fit SVM with chosen value of C
svm_lin = make_pipeline(
    CountVectorizer(max_features=50, stop_words="english", binary=True),
    SVC(kernel='linear', C=0.1)
)

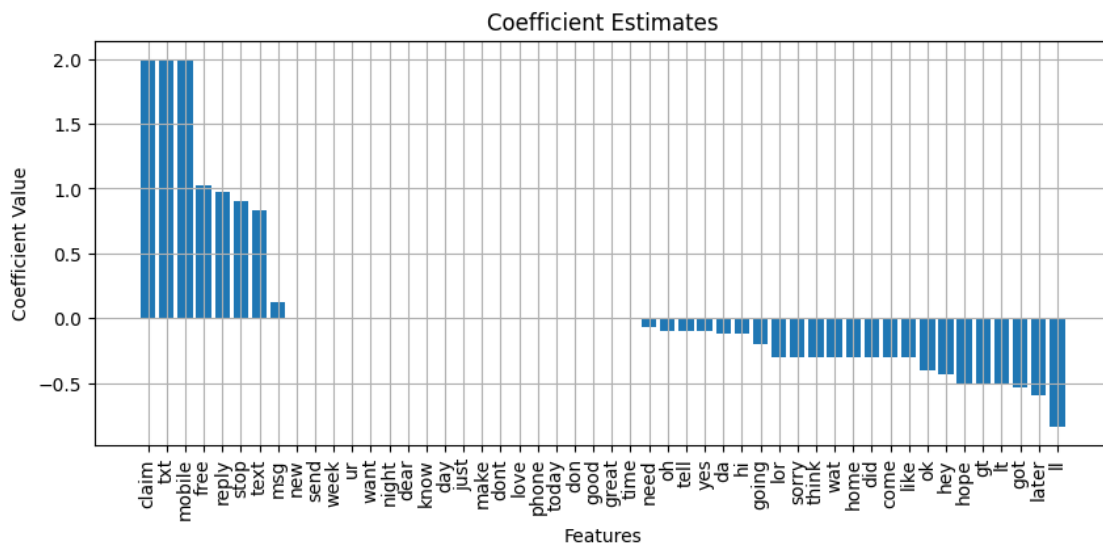
# Fit and tune the model
svm_lin.fit(X_train, y_train)
```

```
[37]: Pipeline(steps=[('countvectorizer',
                        CountVectorizer(binary=True, max_features=50,
                                        stop_words='english')),
                      ('svc', SVC(C=0.1, kernel='linear'))])
```

```
[38]: # Plot the coefficients of the linear SVC
feature_names = svm_lin[-1].get_feature_names_out()
w = svm_lin[-1].coef_.toarray().reshape(-1)

df = pd.DataFrame({'feature': feature_names, 'coef': w}).sort_values ("coef",
↪ascending=False)

plt.figure(figsize=(10,4))
plt.bar(df['feature'], df['coef'])
plt.title('Coefficient Estimates')
plt.xlabel('Features')
plt.ylabel('Coefficient Value')
plt.xticks(rotation=90)
plt.grid()
plt.show()
```



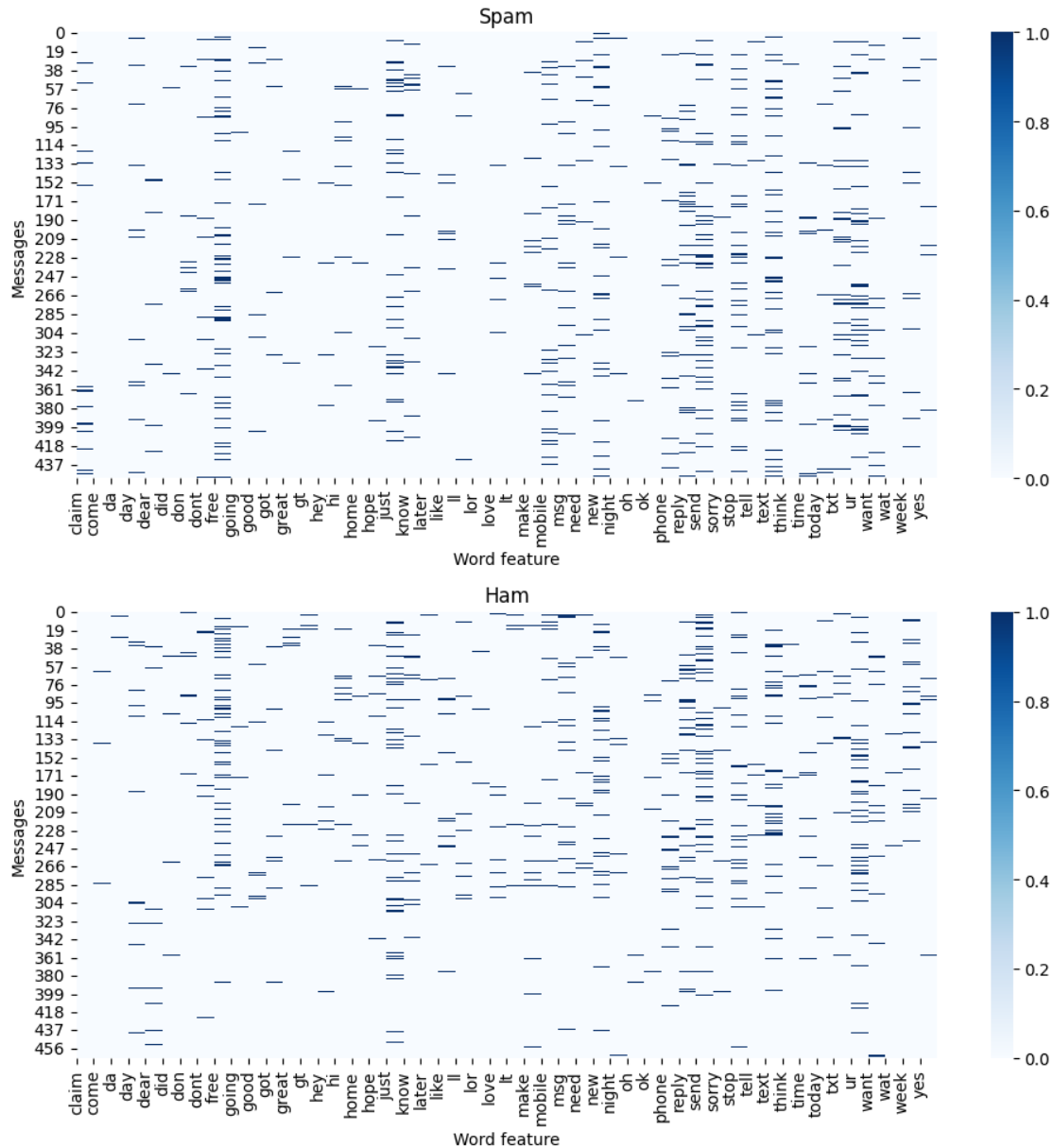
```
[39]: # Draw a heatmap of the support vectors in each class
sv = svm_lin[-1].support_vectors_.toarray() # support vectors
sv_idx = svm_lin[-1].support_ # indices of support vectors
sv_labels = y_train[sv_idx] # labels of support vectors
sv_spam = pd.DataFrame(
    sv[sv_labels == 1,:],
    columns=feature_names)
```

```

)
sv_ham = pd.DataFrame(
    sv[sv_labels == 0,:],
    columns=feature_names
)

fig, ax = plt.subplots(2,1,figsize=(10,10))
sns.heatmap(sv_spam, cmap='Blues', vmin=0, vmax=1, ax=ax[0])
ax[0].set_xlabel('Word feature')
ax[0].set_ylabel('Messages')
ax[0].set_title('Spam')
ax[0].set_xticks(np.arange(len(feature_names)), labels = feature_names)
sns.heatmap(sv_ham, cmap='Blues', vmin=0, vmax=1, ax=ax[1])
ax[1].set_xlabel('Word feature')
ax[1].set_ylabel('Messages')
ax[1].set_title('Ham')
ax[1].set_xticks(np.arange(len(feature_names)), labels = feature_names)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

```



Words like claim, mobile, free, txt, or reply, have positive coefficients and appear more often in spam, indicating messages containing such words are more likely to be classified as spam.

4.0.3 Exercise 11 (CORE)

Now, let's explore using a nonlinear decision boundary: - Alter your pipeline from the previous exercise to use a polynomial kernel of degree 2 with a coefficient of 1. - Tune the parameter C . - Display the confusion matrix on the test data for both models. How do the results compare?

```
[40]: # SVM with polynomial kernel
svm_poly = make_pipeline(
    CountVectorizer(max_features=50, stop_words="english", binary=True),
    SVC(kernel='poly', degree=2, coef0=1)
)

# Grid search over C
C = np.linspace(0.1, 10, 20)
cv_poly = GridSearchCV(
    svm_poly,
    param_grid = {'svc__C': C},
    cv = StratifiedKFold(5, shuffle = True, random_state = 0),
    scoring = ["accuracy", "f1", "recall", "precision"],
    refit='precision' #refit based on accuracy
)

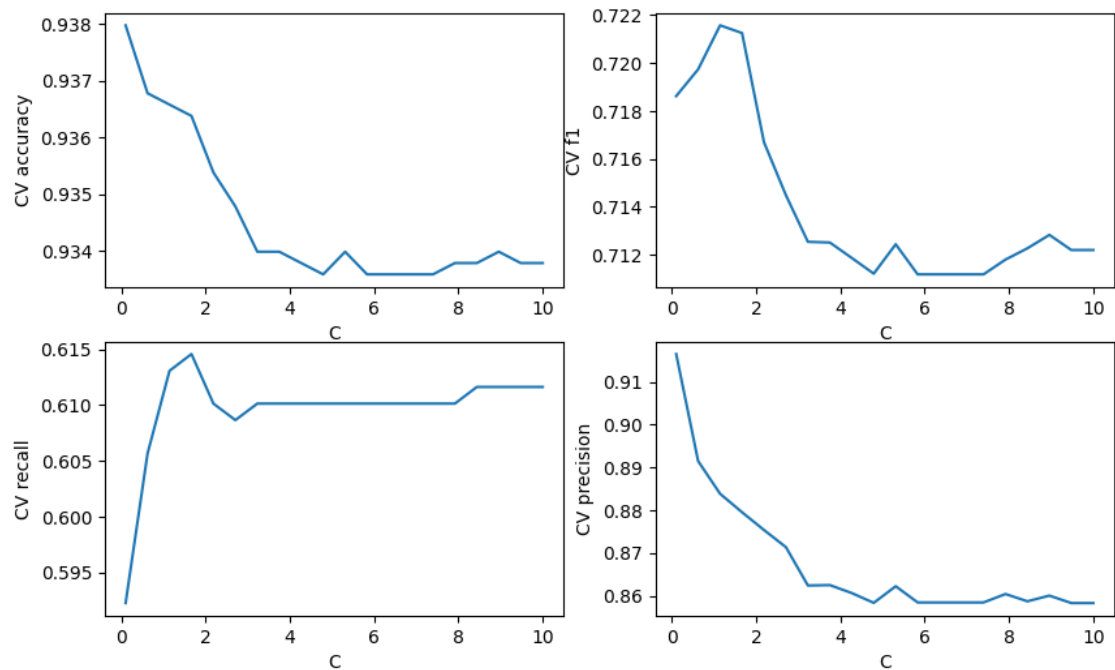
# Fit and tune the model
cv_poly.fit(X_train, y_train)

# Store cv scores in a data frame
cv_accuracy = pd.DataFrame(cv_poly.cv_results_
                           ).
    filter(['param_svc__C', 'mean_test_accuracy', 'mean_test_f1',
           'mean_test_recall', 'mean_test_precision'])
    .rename(columns={'param_svc__C':
                     'C', 'mean_test_accuracy': 'CV accuracy', 'mean_test_f1': 'CV f1',
                     'mean_test_recall': 'CV recall', 'mean_test_precision': 'CV precision'})
```

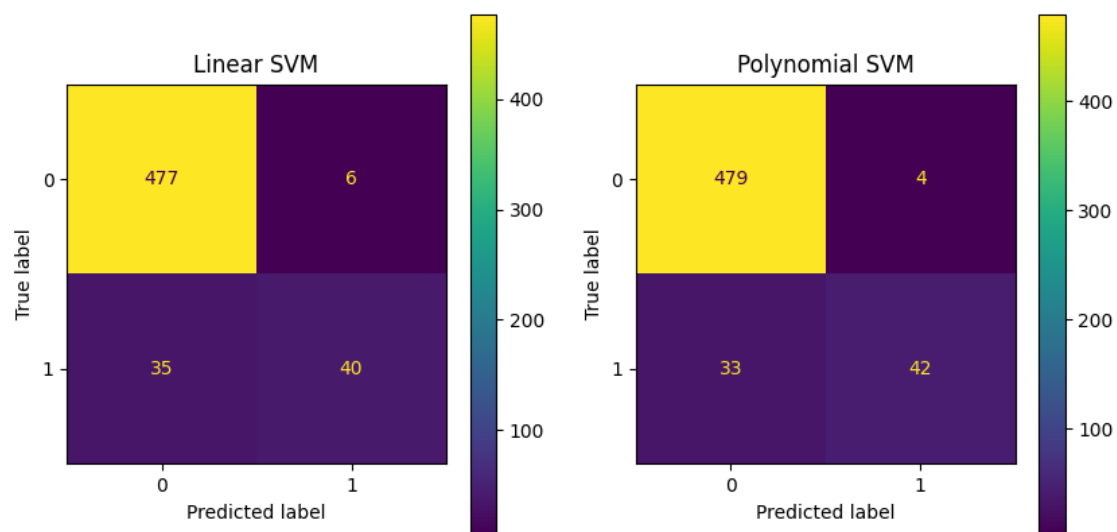
```
[41]: # Get the best model parameters and the recall of the model
print("Params: ", cv_poly.best_params_)
print("Avg Recall: ", cv_poly.best_score_)

# Plot the CV scores
fig, ax = plt.subplots(2,2,figsize=(10,6))
ax = ax.flatten()
sns.lineplot(x='C', y='CV accuracy', data = cv_accuracy, ax=ax[0])
sns.lineplot(x='C', y='CV f1', data = cv_accuracy, ax=ax[1])
sns.lineplot(x='C', y='CV recall', data = cv_accuracy, ax=ax[2])
sns.lineplot(x='C', y='CV precision', data = cv_accuracy, ax=ax[3])
plt.show()
```

```
Params:  {'svc__C': 0.1}
Avg Recall:  0.9165061729231697
```



```
[42]: # Confusion matrix
fig, ax = plt.subplots(1,2,figsize=(10,5))
ConfusionMatrixDisplay.from_estimator(svm_lin, X_test, y_test, ax=ax[0])
ax[0].set_title('Linear SVM')
ConfusionMatrixDisplay.from_estimator(cv_poly.best_estimator_, X_test, y_test,
    ax=ax[1])
ax[1].set_title('Polynomial SVM')
plt.show()
```



The nonlinear decision boundary allows us to identify two additional true positives and two true negatives, resulting in a slight improvement.

4.0.4 Exercise 12 (CORE)

Try increasing the maximum number of features extracted in the bag of words for the linear SVM.
- Visualize the differences between the support vectors for the two classes. Are there any changes to the words which are relevant to distinguishing between the classes? - How do the results compare on the test data?

```
[43]: # SVM with linear kernel
svm_lin2 = make_pipeline(
    CountVectorizer(max_features=100, stop_words="english", binary=True),
    SVC(kernel='linear')
)

# Grid search over C
C = np.linspace(0.1, 10, 20)
cv_lin2 = GridSearchCV(
    svm_lin2,
    param_grid = {'svc__C': C},
    cv = StratifiedKFold(5, shuffle = True, random_state = 0),
    scoring = ["accuracy", "f1", "recall", "precision"],
    refit='precision' #refit based on accuracy
)

# Fit and tune the model
cv_lin2.fit(X_train, y_train)

# Store cv scores in a data frame
cv_accuracy = pd.DataFrame(cv_lin2.cv_results_
                           ).
    filter(['param_svc__C', 'mean_test_accuracy', 'mean_test_f1',
           'mean_test_recall', 'mean_test_precision'])
    .rename(columns={'param_svc__C':
                     'C', 'mean_test_accuracy': 'CV accuracy', 'mean_test_f1': 'CV f1',
                     'mean_test_recall': 'CV recall', 'mean_test_precision': 'CV precision'})

[44]: # Get the best model parameters and the recall of the model
print("Params: ", cv_lin2.best_params_)
print("Avg Accuracy: ", cv_lin2.best_score_)

# Plot the CV scores
fig, ax = plt.subplots(2,2,figsize=(10,6))
ax = ax.flatten()
```

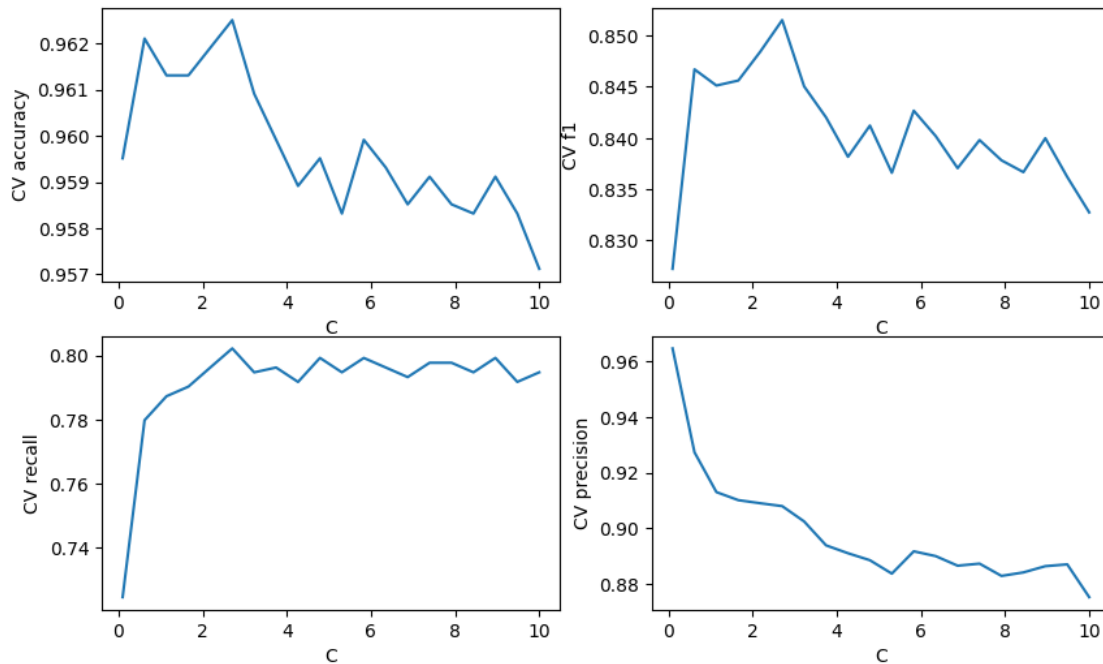
```

sns.lineplot(x='C', y='CV accuracy', data = cv_accuracy,ax=ax[0])
sns.lineplot(x='C', y='CV f1', data = cv_accuracy,ax=ax[1])
sns.lineplot(x='C', y='CV recall', data = cv_accuracy,ax=ax[2])
sns.lineplot(x='C', y='CV precision', data = cv_accuracy,ax=ax[3])
plt.show()

```

Params: {'svc__C': 0.1}

Avg Accuracy: 0.9647769561051375



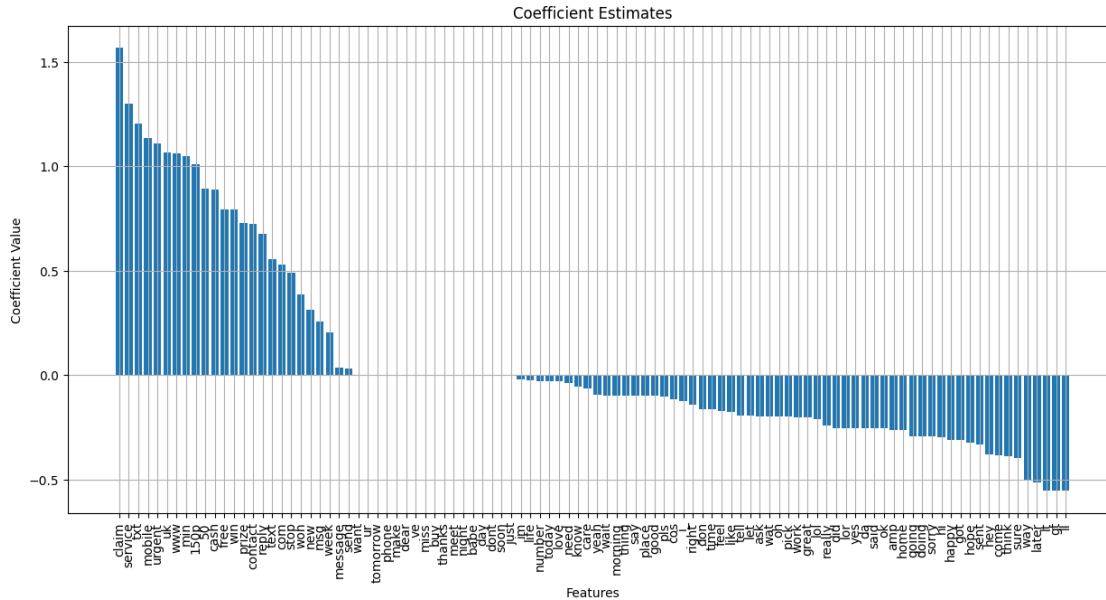
```

[45]: # Plot the coefficients of the linear SVC
feature_names = cv_lin2.best_estimator_[:-1].get_feature_names_out()
w = cv_lin2.best_estimator_[:-1].coef_.toarray().reshape(-1)

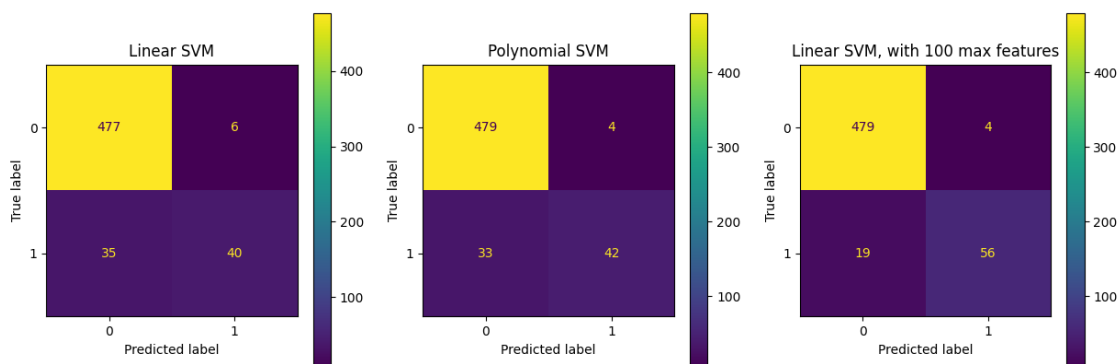
df = pd.DataFrame({'feature': feature_names, 'coef': w}).sort_values ("coef",
↪ascending=False)

plt.figure(figsize=(15,7))
plt.bar(df['feature'], df['coef'])
plt.title('Coefficient Estimates')
plt.xlabel('Features')
plt.ylabel('Coefficient Value')
plt.xticks(rotation=90)
plt.grid()
plt.show()

```



```
[46]: fig, ax = plt.subplots(1,3,figsize=(15,5))
ConfusionMatrixDisplay.from_estimator(svm_lin, X_test, y_test, ax=ax[0])
ax[0].set_title('Linear SVM')
ConfusionMatrixDisplay.from_estimator(cv_poly.best_estimator_, X_test, y_test,
    ↪ax=ax[1])
ax[1].set_title('Polynomial SVM')
ConfusionMatrixDisplay.from_estimator(cv_lin2.best_estimator_, X_test, y_test,
    ↪ax=ax[2])
ax[2].set_title('Linear SVM, with 100 max features')
plt.show()
```



Increasing max features identifies some more words that relevant for distinguishing between spam and ham messages. In particular, we still see differences for words like, claim, mobile, free, txt, and reply, but we find some new words like, service, urgent, uk, and www.

The additional features improve generalization with an additional 16 spam messages identified compared to the linear model with only 50 features.

4.0.5 Exercise 13 (EXTRA)

Lastly, try tuning also the number of features in your CV grid search, with a search over values of `C = np.array([0.1,1,5,10])` and `max_features = np.arange(50,500,10)`. - By comparing the different metrics, which value of `C` and `max_features` would you choose? - Fit the model with your selected values, and display the confusion matrix for the test data. How do the results compare to the previous models?

```
[47]: # SVM with polynomial kernel
svm_lin3 = make_pipeline(
    CountVectorizer(max_features=50, stop_words="english", binary=True),
    SVC(kernel='linear')
)

[48]: # Grid search over C
C = np.array([0.1,1,5,10])
max_features = np.arange(50,500,10)
cv_lin3 = GridSearchCV(
    svm_lin3,
    param_grid = {'svc__C': C, 'countvectorizer__max_features': max_features},
    cv = StratifiedKFold(5, shuffle = True, random_state = 0),
    scoring = ["accuracy", "f1", "recall", "precision"],
    refit='precision' #refit based on accuracy
)

# Fit and tune the model
cv_lin3.fit(X_train, y_train)

[48]: GridSearchCV(cv=StratifiedKFold(n_splits=5, random_state=0, shuffle=True),
    estimator=Pipeline(steps=[('countvectorizer',
                                CountVectorizer(binary=True,
                                                max_features=50,
                                                stop_words='english'))],
                                ('svc', SVC(kernel='linear'))]),
    param_grid={'countvectorizer__max_features': array([ 50,  60,  70,
80,  90, 100, 110, 120, 130, 140, 150, 160, 170,
180, 190, 200, 210, 220, 230, 240, 250, 260, 270, 280, 290, 300,
310, 320, 330, 340, 350, 360, 370, 380, 390, 400, 410, 420, 430,
440, 450, 460, 470, 480, 490]),
    'svc__C': array([ 0.1,  1. ,  5. , 10. ])},
    refit='precision',
    scoring=['accuracy', 'f1', 'recall', 'precision'])

[49]: # Store cv scores in a data frame
cv_accuracy = pd.DataFrame(cv_lin3.cv_results_
```

```

        ).filter(['param_svc__C',
        ↪ 'param_countvectorizer__max_features', 'mean_test_accuracy', 'mean_test_f1',
        ↪ 'mean_test_recall', 'mean_test_precision'])
        ).rename(columns={'param_svc__C':
        ↪ 'C', 'param_countvectorizer__max_features': 'D', 'mean_test_accuracy': 'CV
        ↪ accuracy', 'mean_test_f1': 'CV f1', 'mean_test_recall': 'CV recall',
        ↪ 'mean_test_precision': 'CV precision'})

```

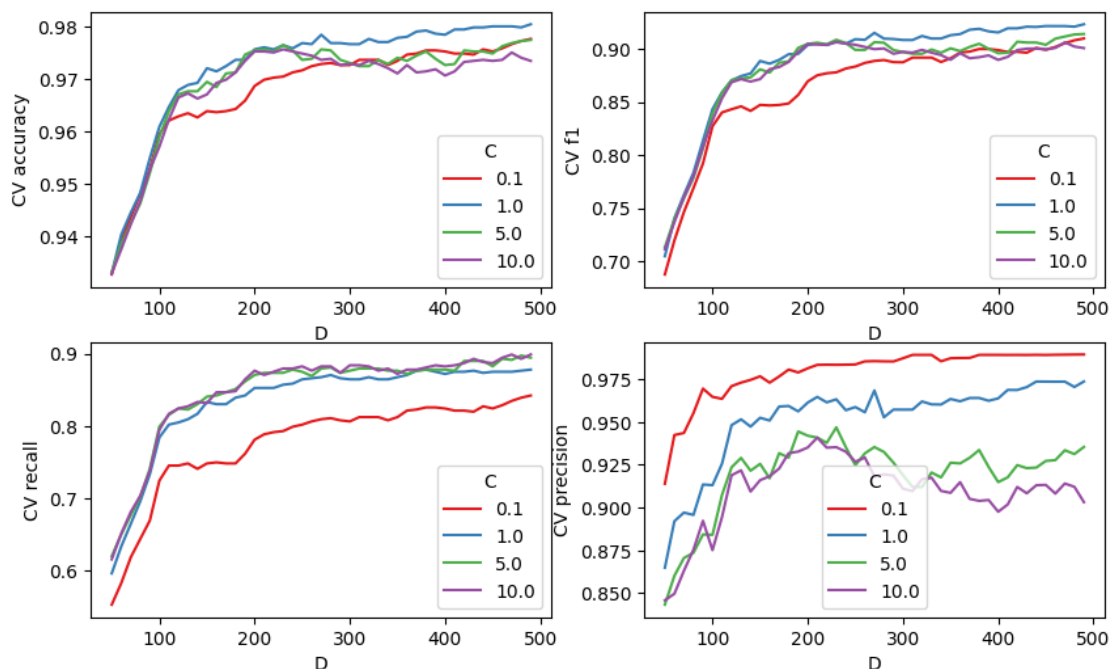
```

[50]: # Get the best model parameters and the recall of the model
print("Params: ", cv_lin3.best_params_)
print("Avg Recall: ", cv_lin3.best_score_)

# Plot the CV scores
fig, ax = plt.subplots(2,2,figsize=(10,6))
ax = ax.flatten()
sns.lineplot(x='D', y='CV accuracy', hue = 'C', palette='Set1', data =
    ↪ cv_accuracy, ax=ax[0])
sns.lineplot(x='D', y='CV f1', hue = 'C', palette='Set1', data =
    ↪ cv_accuracy, ax=ax[1])
sns.lineplot(x='D', y='CV recall', hue = 'C', palette='Set1', data =
    ↪ cv_accuracy, ax=ax[2])
sns.lineplot(x='D', y='CV precision', hue = 'C', palette='Set1', data =
    ↪ cv_accuracy, ax=ax[3])
plt.show()

```

Params: {'countvectorizer__max_features': 490, 'svc__C': 0.1}
 Avg Recall: 0.9895604653251165

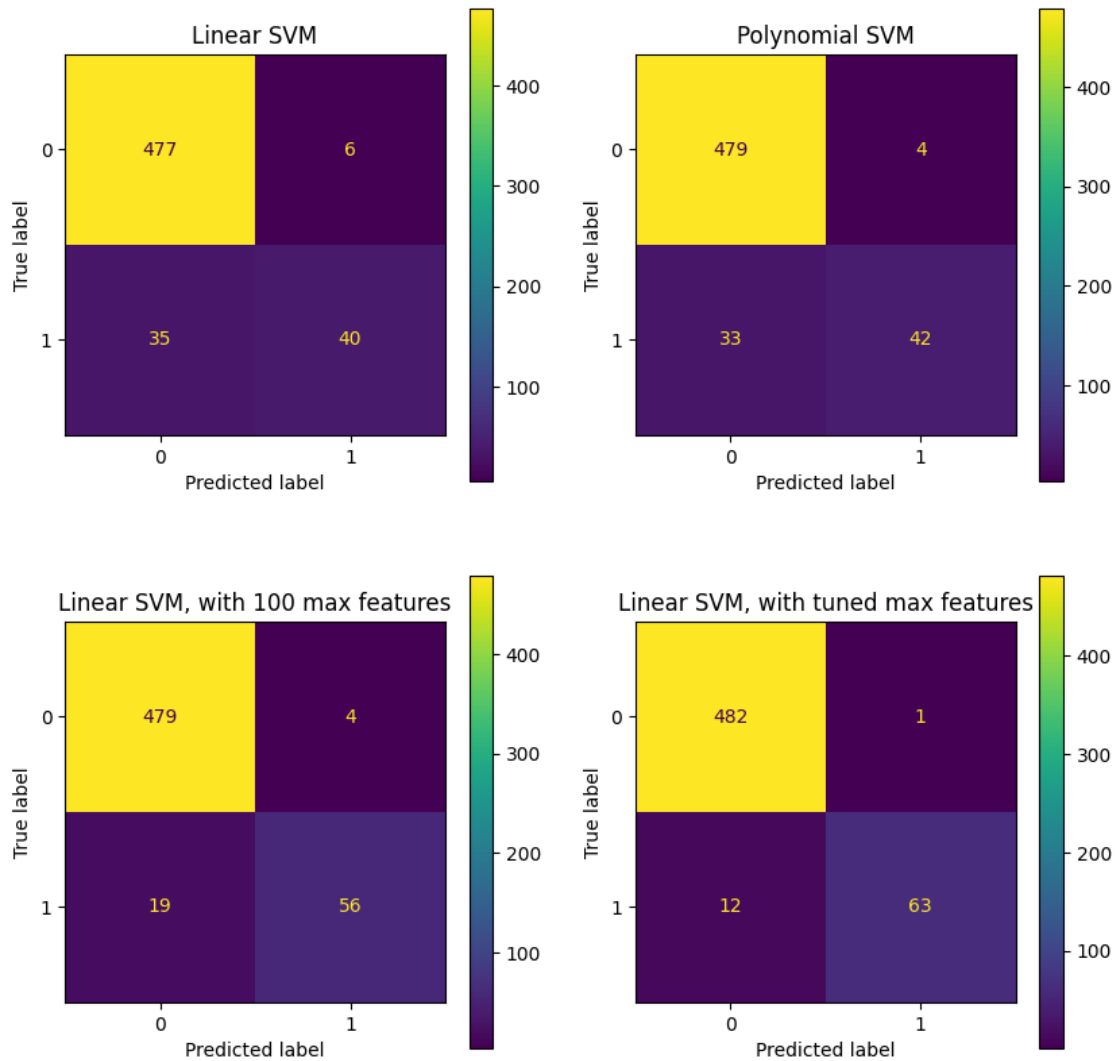


The best estimator selects `C=0.1` and `max_features=480`. However, we observe the precision is fairly constant after about 200 features and the other metrics stabilize after around 300 features. So we may prefer the simpler model with `C=0.1` and less features, e.g. `max_features=300`.

On the other hand, if we don't want to prioritize precision, we may instead prefer `C=1`, which reduces precision slightly but improves accuracy, recall, and f1. Again, the curves become fairly constant after 200-300 features, so we may prefer the simpler model with `C=1` and less features, e.g. `max_features=300`.

```
[51]: # Fit the model with the chosen values
svm_lin3 = make_pipeline(
    CountVectorizer(max_features=300, stop_words="english", binary=True),
    SVC(kernel='linear', C=1)
).fit(X_train, y_train)

[52]: # Confusion matrix
fig, ax = plt.subplots(2,2,figsize=(10,10))
ax = ax.flatten()
ConfusionMatrixDisplay.from_estimator(svm_lin, X_test, y_test, ax=ax[0])
ax[0].set_title('Linear SVM')
ConfusionMatrixDisplay.from_estimator(cv_poly.best_estimator_, X_test, y_test,
    ↪ax=ax[1])
ax[1].set_title('Polynomial SVM')
ConfusionMatrixDisplay.from_estimator(cv_lin2.best_estimator_, X_test, y_test,
    ↪ax=ax[2])
ax[2].set_title('Linear SVM, with 100 max features')
ConfusionMatrixDisplay.from_estimator(svm_lin3, X_test, y_test, ax=ax[3])
ax[3].set_title('Linear SVM, with tuned max features')
plt.show()
```



The results have substantially improved, with only 1 false positive and 12 false negatives.

5 Competing the Worksheet

At this point you have hopefully been able to complete all the CORE exercises and attempted the EXTRA ones. Now is a good time to check the reproducibility of this document by restarting the notebook's kernel and rerunning all cells in order.

Before generating the PDF, please **change 'Student 1' and 'Student 2' at the top of the notebook to include your name(s)**.

Once that is done and you are happy with everything, you can then run the following cell to generate your PDF. Once generated, please submit this PDF on Learn page by 16:00 PM on the Friday of the week the workshop was given.

```
[53]: # !jupyter nbconvert --to pdf mlp_week07_key.ipynb
```